VANTAGE AI

Know Every Token

End-to-End Developer & Product Guidebook

Internal Reference · Engineering · Product Management

Version 1.1 (FIXED) · April 15, 2026

Kamal Soft Pvt Ltd · Confidential

Table of Contents

This guidebook is organized into sixteen chapters plus an appendix. Click any entry below to jump directly to that section. Each chapter also contains links back to related chapters at the end.

Chapter 1. Product Overview & North Star
Chapter 2. Product Strategy & 18-Month Roadmap
Chapter 3. System Architecture
Chapter 4. Data Model — The 18 Tables
Chapter 5. Authentication & Authorization
Chapter 6. Event Ingest Pipeline
Chapter 7. Analytics Engine
Chapter 8. Real-Time: Rate Limits, SSE, Alerts
Chapter 9. Client Integration Patterns
Chapter 10. Cross-Platform Console & Adapters
Chapter 11. Semantic Caching & Waste Detection
Chapter 12. Vantage Agent CLI
Chapter 13. MCP Server & Local Proxy
Chapter 14. Security Model & Audit Findings
Chapter 15. CI/CD, Testing & Operational Runbook
Chapter 16. Competitive Landscape & Moat
Chapter A. Appendix — Glossary, Formulas, API Reference

Chapter 1 — Product Overview & North Star

1.1 What Vantage AI Is

Vantage AI is an AI cost intelligence and observability platform for engineering organizations. It answers a single question that every CTO, Head of Engineering, and CFO is asking in 2026: "We are spending real money on AI coding tools and LLM APIs — is any of it worth it, and can we prove it?"

The product consolidates spend, usage, and quality signals from every place AI is used inside a company into one dashboard: LLM APIs (OpenAI, Anthropic, Google, Mistral), AI coding assistants (GitHub Copilot, Claude Code, Cursor, Gemini CLI, Windsurf, Cline), agent frameworks (LangChain, AutoGen, CrewAI), and the customer's own monitoring stack (Datadog). It attributes every dollar to a developer, a team, a model, and a feature — and layers on anonymized cross-company benchmarks so customers can compare their spend to peers.

> **The one-sentence pitch**
>
> Vantage AI is the only platform that covers the complete AI coding spend stack — IDE tools, LLM APIs, agent frameworks, and existing monitoring — in one dashboard with per-developer attribution and cross-company benchmarks.

1.2 The 10-Year North Star

The long-term position is not "another observability tool." It is the Bloomberg Terminal of AI spend — a trusted, neutral intelligence layer that sits between enterprises and AI vendors, with aggregated data that no single provider will ever have and a fiduciary position no provider can ever claim.

The stages of the journey:

```
Stage 1  Now          Developer Tool
         ACV $0-99     Token tracker, cost dashboard, prompt optimizer
              |
              v
Stage 2  Month 3-6     Engineering Org Platform
         ACV $5K-25K   Multi-tool spend, per-developer ROI, team budgets
              |
              v
Stage 3  Month 9-18    AI Spend Intelligence Layer
         ACV $50K-200K Cross-company benchmarks, vendor negotiation,
                       board-ready audit trails
              |
              v
Stage 4  Year 2-5      Bloomberg of AI Pricing
         ACV $200K-1M  Neutral market intelligence layer,
                       compliance system of record
```

1.3 Why This Is Defensible

Vantage has four permanent structural advantages. None of them are features; they are positions that compound over time.

1. Provider conflict lock-out.

OpenAI cannot tell you Anthropic is cheaper. Anthropic cannot tell you Google outperforms on your workload. No provider will ever show what other companies pay. Vantage is the only party structurally positioned to do this. This is the same principle that let CloudHealth survive AWS for ten years.

2. Data network effect.

Company #500 makes the benchmark data ten times more valuable for companies #1 through #499. More customers means more accurate pricing intelligence, which means better negotiation leverage, which attracts more customers. The flywheel compounds forever.

3. Trusted third party.

When auditors, boards, and regulators ask "how do you govern AI spend?" they will not accept a vendor dashboard. They need an independent system of record. Vantage is building that record right now.

4. Regulatory inevitability.

The EU AI Act, SOX-equivalent AI audit requirements, and HIPAA AI governance are all coming. Compliance tools do not get ripped out. Being the compliance layer is a twenty-year contract, not a monthly subscription.

1.4 Who This Guidebook Is For

Reader	Start at	Focus
Backend engineer joining the team	Chapters 3 → 4 → 6	Architecture, data model, ingest pipeline
Frontend / dashboard engineer	Chapters 3 → 7 → 10	Request flow, analytics endpoints, cross-platform tab
Product manager	Chapters 1 → 2 → 16	Strategy, roadmap, competitive position
DevRel / solutions engineer	Chapters 9 → 13 → 12	Client integrations, MCP, CLI agent
Security reviewer	Chapters 5 → 14	Auth, session cookies, audit findings
On-call engineer	Chapter 15	Operational runbook, deploys, KV ops

1.5 Reading Conventions

- **Code blocks** are monospaced with a blue rule on the left. They contain real SQL, TypeScript, Python, and shell commands from the codebase.
- **Callouts** in blue boxes are essential context. Yellow boxes are warnings. Green boxes are tips. Red boxes are things that will break production.

- **Tables** summarise decision matrices, API surfaces, and competitive comparisons. Always scan these first before reading the prose around them.
- **ASCII diagrams** show data flow and sequence. They are intentionally simple so you can reproduce them on a whiteboard in a meeting.

Continue to **Chapter 2 — Product Strategy & 18-Month Roadmap**

Chapter 2 — Product Strategy & 18-Month Roadmap

2.1 Honest Diagnosis

The platform is built. The feature set covers the entire AI coding spend stack — IDE tools, LLM APIs, agent frameworks — in a single dashboard with cross-company benchmark intelligence. No funded competitor does all of this. The P2 milestone (Copilot adapter, Datadog exporter, benchmark system, cross-platform console) is complete as of April 2026. The real work now is distribution.

What is working:

Area	Signal
OTel collector for AI coding tools	Genuinely unique — no competitor tracks Claude Code + Gemini CLI + Copilot + Cursor in one pipeline
GitHub Copilot Metrics API adapter	First mover; API went GA in February 2026
Anonymized benchmark system	k-anonymity, quarterly snapshots, percentile rankings — the data moat compounding
Cross-Platform Console	Full per-developer spend across OTel + Copilot billing + Datadog
Datadog exporter	Meets customers where monitoring already lives — procurement unlock
MCP server (12 tools)	Right timing, daily-driver behavior in Claude Code, Cursor, Windsurf
CLI wrapper (npx vantageai-cli)	Transparent AI agent wrapper with viral potential
Trust page + privacy modes	Three privacy modes and zero-interception positioning — enterprise procurement differentiator

What is fragile:

Area	Problem
Distribution	Platform is built but nobody knows. Every week without distribution is a week a funded competitor can catch up
No social proof	Testimonials correctly removed, but no logos, no case studies, no named customers — enterprise sales stalls
No paying customers	All metrics are product metrics; revenue is the only external signal that matters
Brand name	"VantageAI" trademark conflict risk; vantageaiops.com is clunky in enterprise sales
Palma.ai	Direct ICP competitor; still pre-PMF but watch closely
Benchmark seed	k-anonymity floor of 5 orgs means most cohorts return 404 until more orgs opt in — chicken-and-egg problem

2.2 The Existential Threat

> **The 12-month shelf life**
>
> OpenAI launched Usage Dashboard in Q4 2024. Anthropic Console has spend analytics. Google Cloud AI has cost monitoring. All free, all improving. The "show costs" story has a twelve-month shelf life before providers make it redundant. The only response is to build what providers are structurally prevented from building: cross-provider, cross-company intelligence.

2.3 The Pivot — Tool to Platform

The pivot is not a rebuild. It is a reframe: every feature already built is still valid, but the narrative and the target buyer need to shift. The Stage 1 developer is still a valid user, but the ICP is now the CTO at a fifty-to-five-hundred-person engineering organization.

Stage	Target buyer	ACV	Positioning
1 — Now	Individual developers	$0-99/mo	Token tracker + cost dashboard
2 — Month 3-6	CTOs at 50-500 companies	$5K-25K/yr	Engineering org platform
3 — Month 9-18	CFOs, procurement, VPs	$50K-200K/yr	AI spend intelligence layer
4 — Year 2-5	Enterprise + regulators	$200K-1M/yr	Bloomberg of AI pricing

2.4 Historical Analogs Worth Studying

Company	What They Did	Outcome	Vantage Parallel
CloudHealth	Multi-cloud cost intelligence vs AWS/Azure/GCP	$500M VMware acquisition	Multi-model AI spend vs OpenAI/Anthropic/Google
Apptio	IT financial management, tech spend transparency	$1.94B IBM acquisition	AI financial management + ROI
Bloomberg	Neutral market data terminal — banks cannot self-provide	$10B+ private	AI model pricing terminal
Gartner	Procurement intelligence — vendors cannot self-report	$5B+ public	AI procurement intelligence

2.5 The 18-Month War Roadmap

Phase 1 — Fix the Foundation (Weeks 1–6, COMPLETE)

Goal: 500 active accounts, 5 design partner CTOs, $1K MRR, one HN front page.

✅ CI signup rate-limit bypass (PR #53) - Commit c08f5c5 (2026-04-14) [NEWLY ADDED]

- Free tier raised to 50K OTel events/month (PR #54)
- Fake testimonials removed; honest design-partner CTA shipped (PR #55)

- Benchmark opt-in toggle added to settings page (PR #55)
- OTel developer.id extraction bug fixed (PR #52)
- AI Spend Console cross-platform MVP shipped (PR #51)
- Copilot Metrics API adapter shipped, AES-256-GCM encrypted (PR #55)
- Trust page, report page, enterprise tier, audit log — all live

Phase 2 — Build the Enterprise Wedge (Months 2–4)

Goal: 3 paying enterprise accounts ($1K-5K/yr), $5K MRR, benchmark report hits 2K shares.

- Semantic cache layer — Cloudflare Workers + Vectorize, configurable threshold
- Prompt registry MVP — versions, cost-compare, deploy via MCP
- First benchmark report: "State of AI Coding Tool Spend Q2 2026"
- Seed benchmark data via 5 design partner orgs opting in
- Slack integration — native app beyond the existing webhooks
- n8n automation deployment on Railway — onboarding drip and conversion workflows
- SOC 2 prep with Vanta — start Month 3, target completion Month 7–8

Phase 3 — Activate the Intelligence Layer (Months 5–8)

Goal: 10 enterprise accounts, $25K MRR, first $50K ACV deal, Series A narrative ready.

- Benchmark dashboard — "your cost/token vs industry median" across 6 model categories
- Vendor negotiation module — "here is what similar companies paid at their last Copilot renewal"
- Quality-adjusted routing engine — route by task type from historical quality + cost data
- AI governance report — auto-generate board-ready AI spend report
- Agent trace DAG visualization — graph view of multi-step agent sessions from OTel data
- Quarterly "AI Spend Index" public report — PR strategy, builds brand authority
- SOC 2 Type I certification

Phase 4 — Become the Standard (Months 9–18)

Goal: $100K MRR, 50 enterprise accounts, system of record at 3 public companies.

- AI Procurement API — Ramp, Brex, Coupa pull spend data directly
- Compliance module — EU AI Act audit trail, HIPAA AI governance
- Model Performance Index — public quality/cost ranking per task type
- Series A raise with 10+ enterprise customers and benchmark data from 500+ companies
- First BD hire — founder builds, BD sells

2.6 Financial Model (Targets)

Tier	Price	Month 12 target	MRR	Gross margin
Free	$0	Unlimited	$0	—
Team	$99/mo	50	$4,950	~85%
Business	$499/mo	20	$9,980	~80%

Tier	Price	Month 12 target	MRR	Gross margin
Enterprise	$2K-10K/mo	5	$20K-50K	~75%
Total			$35K-65K MRR	~80%

Monthly burn is $300–900/mo at current scale (Cloudflare, Render, Anthropic API for quality scoring, n8n, domains, tools). Break-even: 4–10 Team plan customers. Ramen-profitable at roughly $2K MRR.

2.7 Open Gaps (as of April 2026)

ID	Gap	Severity	Fix owner
GAP 1	No benchmark data seeded — k-anonymity floor returns 404	Critical	GTM: design partner outreach
GAP 10	No paying customers yet	Critical	GTM: 5 design partner CTOs
GAP 4	No AI gateway / semantic caching shipped	Major	Eng: Month 2
GAP 5	No prompt management / versioning	Major	Eng: Month 3-4
GAP 6	Brand/domain fragmentation ("vantageaiops.com" is clunky)	Minor	Founder: Week 1
GAP 7	No enterprise social proof — no logos, no case studies	Minor	GTM: ongoing

---

Related: [Chapter 16 — Competitive Landscape & Moat](#) for the full competitor matrix.

Chapter 3 — System Architecture

3.1 Technology Stack

Vantage is deliberately built on a minimal serverless stack so one engineer can operate the whole thing. There are no containers, no Kubernetes, no dedicated database servers. Everything runs at the edge on Cloudflare.

Layer	Technology	Why
API Worker	Cloudflare Workers + Hono	Edge globally distributed, zero cold starts, TypeScript
Database	Cloudflare D1 (SQLite)	Serverless SQLite, no infra, free tier sufficient for MVP
Cache / pub-sub	Cloudflare KV	Rate limiting, SSE broadcast, alert throttle, session tokens, secrets
Frontend	Cloudflare Pages	Static hosting, global CDN, auto-deploys from main
Email	Resend API	Transactional email, 3K/month free, custom domain
SDK (Python)	vantageaiops on PyPI	OpenAI + Anthropic proxy wrappers
SDK (JS)	vantageaiops on npm	OpenAI + Anthropic wrappers with streaming support
MCP server	vantageaiops-mcp on npm	Claude Code, Cursor, Windsurf integration — 12 tools
CLI agent	npx vantageai-cli	Transparent AI agent wrapper (Python 3.11+)
CI/CD	GitHub Actions	Deploy on push to main, test on every branch

3.2 High-Level Data Flow

Every LLM call observed by Vantage — whether from an SDK, the OTel collector, the MCP tool, the local proxy, or a direct API POST — lands at one Cloudflare Worker, passes through one middleware pipeline, and is stored in one D1 database. The dashboard polls the same API the ingestion pipeline writes to. There is no separate read replica, no message queue, no cache tier to invalidate.

```
SDK / Direct API / OTel / MCP / Copilot poller / Datadog push
                            |
                            v
             Bearer vnt_... token
             POST /v1/events  (or /v1/otel/v1/metrics, etc.)
                            |
                            v
 +-----------------------------------------------+
 |           Cloudflare Worker                   |
 |        api.vantageaiops.com                   |
 |                                               |
```

```
|  corsMiddleware -> authMiddleware              |
|              -> rate limiter (KV)              |
|              -> route handler                  |
|                                                |
|  events.ts  -> D1 INSERT                       |
|             -> KV broadcast stream:{orgId}     |
+------------------+------------+----------------+
                   |            |
                 D1            KV
        (events, orgs,    (rl:, stream:,
         sessions,         sse:, slack:,
         members,          alert:, recover:)
         budgets,
         alerts, ...)
                   |
              +----+----+
              |         |
          Dashboard   SSE stream
          (polling    GET /v1/stream/:orgId
           /analytics)(KV polling every 2s)
```

3.3 Request Lifecycle

Every authenticated request goes through the same seven-step pipeline. Understanding this is essential before debugging any production issue.

1. CORS preflight — corsMiddleware returns 204 with CORS headers if the request is OPTIONS.
2. Auth resolution — authMiddleware tries the cookie path first (vantage_session → sessions table), then the Bearer path (Authorization: Bearer vnt_... → SHA-256 hash → orgs or org_members table). Sets context variables orgId, role, scopeTeam, memberId.
3. Rate limit — KV fixed-window counter at key rl:{orgId}:{minuteBucket}. Default limit is 1,000 requests per minute per org, configurable via RATE_LIMIT_RPM. Returns 429 with Retry-After on breach.
4. Role guard — routes requiring admin check adminOnly; ingestion routes block viewer role inline (not middleware).
5. Route handler business logic — the specific handler for the endpoint.
6. D1 read/write — always includes WHERE org_id = ? for tenant isolation.
7. KV side effects — broadcast to SSE channel, update cache, fire async alert, etc.

3.4 Infrastructure Bindings

The Worker is configured via wrangler.toml. Secrets are set via wrangler secret put and never committed. The database_id and KV namespace id live in a gitignored section of wrangler.toml.

```
# Worker name
name = "vantageai-api"
routes = [
  { pattern = "api.vantageaiops.com/*", zone_name = "vantageaiops.com" }
]

# D1 SQLite — binding name only
[[d1_databases]]
binding = "DB"
database_name = "vantageai-production"
# database_id = "..."  (set locally, gitignored)

# KV Namespace — binding name only
[[kv_namespaces]]
binding = "KV"
# id = "..."  (set locally, gitignored)

# Non-secret env vars
[vars]
ENVIRONMENT = "production"
ALLOWED_ORIGINS = "https://vantageaiops.com,https://www.vantageaiops.com"
RATE_LIMIT_RPM = "1000"

# Secrets (set via: wrangler secret put)
#   RESEND_API_KEY        — email sending
#   COPILOT_ENC_KEY       — AES-256-GCM for GitHub Copilot tokens
#   DATADOG_ENC_KEY       — AES-256-GCM for Datadog API keys
```

3.5 The Vantage Intelligence Engine (VIE)

Every event Vantage sees passes through the same nine-stage pipeline. No event bypasses enrichment. No event skips benchmarking. The intelligence compounds because every event makes the model better.

```
[SDK / CLI / OTel / MCP / Copilot API / Datadog Push]
         |
         v
[01 INGEST]       validate, auth, rate-limit (1K RPM/org via KV)
         |
         v
[02 NORMALIZE]    VantageEvent schema, provider detection,
                  model canonicalization
         |
```

```
         v
[03 ENRICH]       live pricing injection (MODEL_PRICES),
                  org context, developer attribution, team tagging
         |
         v
[04 SCORE]        efficiency score 0-100, prompt optimizer analysis
         |
         v
[05 SEMANTIC CACHE CHECK]  Cloudflare Vectorize cosine similarity
                           (default 0.92)
    cache HIT ----------------> RETURN cached response + cost saved
    cache MISS |
               v
[06 BENCHMARK]    compare to anonymized cohort percentiles
                  (opt-in orgs only, k>=5)
         |
         v
[07 ALERT ENGINE] Z-score anomaly, budget threshold,
                  quality regression checks
    alert ---------------------> Slack webhook / email (async)
         |
         v waitUntil()  — non-blocking async fan-out
[08 STORE]        D1 (events + otel_events + cross_platform_usage
                     + audit_events)
         |
         v
[09 SERVE]        SSE push -> dashboard | MCP tool response | API JSON
```

Architecture principle

Every token Vantage sees passes through the same pipeline. No event bypasses enrichment. No event skips benchmarking. The intelligence compounds because every event makes the model better.

3.6 Background Jobs and Cron Triggers

Job	Trigger	What it does
Copilot sync	Cron — Sundays 00:00 UTC	Pulls GitHub Copilot Metrics API for all connected orgs, upserts into cross_platform_usage
Datadog push	Cron — hourly	Pushes vantage.ai.cost_usd and vantage.ai.tokens gauge metrics to customer Datadog sites
Benchmark rollup	Cron — quarterly	Computes p25/p50/p75/p90 per cohort if sample_size ≥ 5
Anomaly detection	Cron — every 10 min	Z-score on rolling 7-day hourly cost, sends Slack alert if > 3 sigma
Session cleanup	Cron — daily 03:00	Deletes expired sessions from D1

Job	Trigger	What it does
	UTC	
Repo backup	Cron — daily 03:00 UTC + push to main	Mirror to Amanjain98/Vantage-AI + ZIP artifact

---

Next: **Chapter 4 — Data Model** walks through all eighteen D1 tables and the relationships between them.

Chapter 4 — Data Model: The 18 Tables

4.1 Schema Overview

🔧 FIXED v1.1: Complete migration history now catalogued in new Section 4.6 with all 14 migrations documented (dates, purposes, file references)

Vantage uses Cloudflare D1, which is serverless SQLite. The production database has eighteen tables and fourteen applied migrations. All timestamps are stored as INTEGER Unix epoch seconds — with one exception: cross_platform_usage uses TEXT in the format YYYY-MM-DD HH:MM:SS because OTel semantic conventions send timestamps that way, and normalizing at write time would lose information.

> **Tenant isolation is enforced at the query layer**
>
> Every D1 query in the codebase includes WHERE org_id = ? bound to the authenticated org_id. There is no query in the entire worker that returns data across multiple orgs. For scoped members, a teamScope() helper appends AND team = ? to every analytics query — so a viewer with scope_team="backend" can never see team="frontend" data, enforced at the SQL level, not application level.

4.2 Table Reference

Identity & Access

Table	Key fields	Purpose
orgs	id, api_key_hash, plan, budget_usd, benchmark_opt_in	One row per organization (account owner). Stores the SHA-256 hash of the owner API key, plan tier, monthly budget, and whether the org has opted in to anonymized benchmarks.
org_members	role, scope_team, api_key_hash	Team members under an org. Each member has their own API key hash. role is owner, admin, member, or viewer. scope_team scopes their data view to one team.
sessions	token, org_id, role, expires_at	HTTP-only cookie sessions. 30-day TTL. 256-bit entropy token. Also stores a short-lived sse_token used by the SSE endpoint.

Event Ingestion

Table	Key fields	Purpose
events	event_id, provider, model, tokens, cost_usd, team, trace_id, prompt_hash, cache_hit, quality scores	Core data table — one row per LLM call. Also stores agent trace fields (trace_id, parent_event_id, span_depth, agent_name) and quality scores written back asynchronously.
otel_events	provider, session_id, developer_email, model, cost_usd	Raw OpenTelemetry events from the AI coding tool collector. Powers timeseries and model

Table	Key fields	Purpose
		breakdown endpoints.
cross_platform_usage	developer_id, tool_type, source, period_start, period_end	Normalized multi-tool spend per developer, per period. Populated by both OTel ingest and Copilot billing sync. This is the canonical table for per-developer cross-tool attribution.

Governance & Alerts

Table	Key fields	Purpose
team_budgets	org_id, team, budget_usd	Per-team monthly budget limits. Joined into team analytics queries to show budget percentage per team.
alert_configs	slack_url, trigger_budget, trigger_anomaly	Slack webhook URL and which alert types are enabled for this org. Upsert pattern — re-calling /v1/alerts/slack updates the row.
budget_policies	org_id, threshold_pct, action	Graduated alert thresholds (50%, 75%, 85%, 100%) per team and per org.
audit_events	org_id, actor, event_type, payload, created_at	Admin action log — every member invite, key rotation, config change, budget update. Indexed by (org_id, created_at DESC).

Cross-Platform Adapters

Table	Key fields	Purpose
provider_connections	org_id, provider, status	Generic connection status table shared across Copilot, Datadog, and future adapters.
copilot_connections	org_id, kv_key, last_sync	GitHub Copilot token metadata. The actual token is AES-256-GCM encrypted in KV (never in D1); this table stores the KV key reference and last sync timestamp.
datadog_connections	org_id, kv_key, site, last_push	Datadog API key metadata. AES-256-GCM encrypted in KV. Stores the customer Datadog site (datadoghq.com, datadoghq.eu, etc.).

Benchmark Intelligence

Table	Key fields	Purpose
benchmark_cohorts	org_id, size_band, industry, opt_in	Opt-in cohort assignments. Size bands are 1–10, 11–50, 51–200, 201–1000, 1000+.
benchmark_snapshots	cohort_id, period, metric, p25/p50/p75/p90	Quarterly anonymized metrics per cohort. k-anonymity enforced: a snapshot is only written

Table	Key fields	Purpose
		if sample_size ≥ 5.
benchmark_contributions	org_id, snapshot_id, contributed_at	Join table tracking which orgs contributed to which snapshots. org_id never appears in snapshot rows themselves — only in this join table — so anonymized queries cannot be de-anonymized.

Platform Analytics

Table	Key fields	Purpose
platform_pageviews	page, referrer, ts	Anonymous landing page analytics.
platform_sessions	session_id, pages_viewed, duration	Anonymous session-level analytics.

4.3 The events Table in Detail

The events table is the core of the product. Every analytics query — cost, tokens, models, teams, traces, KPIs — aggregates it. Understanding the columns is essential.

```
CREATE TABLE events (
  id              TEXT PRIMARY KEY,  -- event_id (UUID)
  org_id          TEXT NOT NULL,
  team            TEXT,              -- optional tag for team scoping
  user_id         TEXT,              -- SHA-256 hashed user identifier
  project         TEXT,
  feature         TEXT,

  provider        TEXT NOT NULL,     -- openai, anthropic, google, ...
  model           TEXT NOT NULL,     -- gpt-4o, claude-sonnet-4-6, ...
  prompt_tokens   INTEGER DEFAULT 0,
  completion_tokens INTEGER DEFAULT 0,
  cache_tokens    INTEGER DEFAULT 0, -- provider native cache reads
  total_tokens    INTEGER DEFAULT 0,
  cost_usd        REAL    DEFAULT 0, -- always canonical; SDK aliases
                                     -- (total_cost_usd, cost_total_usd)
                                     -- are normalized by buildInsertStmt
  latency_ms      INTEGER DEFAULT 0,

  -- Agent tracing
  trace_id        TEXT,
  parent_event_id TEXT,
  span_depth      INTEGER DEFAULT 0,
```

```
  agent_name      TEXT,

  -- Semantic cache / dedup
  prompt_hash     TEXT,              -- SHA-256 first 16 hex chars
  cache_hit       INTEGER DEFAULT 0, -- 1 if duplicate of recent prompt

  -- Quality scores (written back async by LLM judge)
  hallucination_score REAL,
  faithfulness_score  REAL,
  relevancy_score     REAL,
  consistency_score   REAL,
  toxicity_score      REAL,
  efficiency_score    REAL,          -- 0-100 composite

  created_at      INTEGER NOT NULL   -- unix epoch seconds
);
CREATE INDEX idx_events_org_created ON events(org_id, created_at DESC);
CREATE INDEX idx_events_org_team    ON events(org_id, team);
CREATE INDEX idx_events_trace       ON events(org_id, trace_id);
CREATE INDEX idx_events_prompt_hash ON events(org_id, prompt_hash)
  WHERE prompt_hash IS NOT NULL;
```

4.4 Relationships (simplified ERD)

```
          orgs
           |  1
           |
           | *                *           *            *
       org_members  ---- events ---- team_budgets ---- audit_events
           |                  |
           | *                | *
       sessions          otel_events
                             |
                             | *
                     cross_platform_usage
                             |
                      +-------+--------+
                      |                |
               copilot_connections   datadog_connections
                      |                |
                     (KV)             (KV)
                   AES-256-GCM      AES-256-GCM

           orgs -- benchmark_cohorts -- benchmark_contributions
```

```
                                                |
                                                | *
                                        benchmark_snapshots
                                        (p25/p50/p75/p90)
```

4.5 SDK Field Normalization

Different SDK versions use different field names. The ingest handler accepts all variants and normalizes to the canonical form before INSERT. This is the mapping:

Canonical field	SDK aliases accepted
event_id	id
prompt_tokens	usage_prompt_tokens
completion_tokens	usage_completion_tokens
cache_tokens	usage_cached_tokens, cache_tokens
total_tokens	usage_total_tokens (or prompt+completion sum)
total_cost_usd	cost_total_usd, cost_total_cost_usd, cost_usd

Next: Chapter 5 — Authentication & Authorization. Before any event reaches this table, auth middleware has to resolve the identity.

Chapter 5 — Authentication & Authorization

5.1 API Key Format

Vantage uses a namespace-prefixed API key scheme that encodes the org id into the key itself. This allows the worker to route a request to the correct tenant without a database lookup, while still providing 128 bits of cryptographic entropy.

```
vnt_{orgId}_{16-hex-random}
 ^     ^          ^
 |     |          +-- 16 bytes of crypto.getRandomValues()
 |     |              = 128 bits of entropy
 |     +------------- org slug (for fast routing)
 +------------------- VantageAI namespace prefix

Example:  vnt_acme_a1b2c3d4e5f67890
```

Only the SHA-256 hash of the raw key is stored in D1. The raw key is shown once in the dashboard (and once in the member-invite email) and can never be retrieved afterwards. A twelve-character hint ("vnt_acme...") is stored so the UI can show which key is currently active.

> **Rotation is instant**
>
> There is no grace period for an old key — rotating invalidates the old hash immediately. Always update the key in every deployed environment before rotating, or automate the rotation through a secret manager.

5.2 Auth Middleware Flow

Every request is resolved against two auth paths: the cookie path (for dashboard users) and the Bearer path (for SDK/CLI/direct API callers). The cookie path is tried first because it is cheaper — a single sessions table lookup — and sessions are created through an authenticated bearer flow anyway.

```
Request arrives
      |
      +-- Has Cookie: vantage_session=TOKEN ?
      |       |
      |       +-- YES -> SELECT FROM sessions
      |       |          WHERE token=? AND expires_at > now()
      |       |          -> If found: set orgId, role,
      |       |             memberId, scopeTeam
      |       |          -> If not found: fall through
      |       |
```

```
      |       +-- NO -> continue
      |
      +-- Has Authorization: Bearer vnt_... ?
      |       |
      |       +-- YES -> Extract orgId from key format (parts[1])
      |       |          -> SHA-256 hash the key
      |       |          -> Check orgs table (owner key)
      |       |          -> If not found, check org_members
      |       |          -> If still not found in prod: 401
      |       |          -> In dev: auto-create org (convenience)
      |       |
      |       +-- NO -> 401 "Missing or invalid API key"
      |
      +-- Rate limit check -> 429 if exceeded
```

5.3 Role Hierarchy

Role	Source	Permissions
owner	orgs.api_key_hash	All actions. Only owner can rotate the root key via POST /rotate.
admin	org_members.role='admin'	Member management, team budgets, admin overview, alert config.
member	org_members.role='member'	Write events, read analytics. No admin endpoints.
viewer	org_members.role='viewer'	Read-only. Blocked from POST /v1/events with inline 403.

Role checks are enforced with two helpers:

- **adminOnly** — middleware that returns 403 unless role is owner or admin. Applied to member management, admin overview, and team budget endpoints.
- **viewer block** — inline guard in POST /v1/events that returns 403 for viewer role. Not a middleware because only one endpoint needs it.
- **teamScope()** — appends AND team = ? to every analytics query when the authenticated member has a scope_team set. This is the mechanism that makes team-scoped viewers actually see only their team's data.

5.4 Session Security Properties

Property	Value	Reason
Cookie flags	HttpOnly; SameSite=Lax; Secure	XSS protection, CSRF protection, HTTPS-only
Domain	vantageaiops.com (prod only)	Shared across api. and app. subdomains
TTL	30 days	Owner can rotate root key to invalidate all sessions

Property	Value	Reason
Token entropy	256 bits (64 hex chars)	Unguessable by brute force
Storage	D1 sessions table	D1 provides consistent expiry and deletion (KV is eventual)

5.5 Key Recovery Flow

The recovery flow is carefully designed around a single operational constraint: email scanners (Gmail Safe Links, Outlook ATP) automatically follow GET links in emails before the user ever clicks them. If a recovery token were consumed on GET, the user's recovery link would always be dead by the time they clicked it.

```
POST /v1/auth/recover { email }
        |
        +-- Always returns 200 (don't leak email existence)
        |
        +-- If email in orgs:
        |     -> Generate 48-char hex token (24 random bytes)
        |     -> KV.put("recover:{token}",
        |               {orgId, type:'owner'},
        |               TTL=3600s)
        |     -> Build redeem URL:
        |        api.vantageaiops.com/v1/auth/recover/redeem?token=...
        |     -> Send email via Resend
        |
        +-- If email in org_members:
              -> Send hint-only email — admins reissue member keys

Email link clicked
        |
        v
GET /v1/auth/recover/redeem?token=TOKEN
        |
        +-- "Peek" at KV (do NOT delete)
        +-- If valid   -> redirect /auth?confirm_token=TOKEN
        +-- If invalid -> redirect /auth?recovery_error=expired

User clicks "Get new key" button
        |
        v
POST /v1/auth/recover/redeem { token }
        |
        +-- KV.get("recover:{token}") -> verify
```

```
        +-- KV.delete("recover:{token}") -> consume (single-use)
        +-- Generate new key -> hash -> UPDATE orgs
        +-- Return { ok: true, api_key: newKey, hint }
```

Why GET does not consume

GET requests are idempotent. Email scanners hit every GET link in the email. Only POST /v1/auth/recover/redeem consumes the token — GET is a "peek and redirect" that tells the user whether the token is still valid.

Next: **Chapter 6 — Event Ingest Pipeline**. How events get from the SDK into the events table.

Chapter 6 — Event Ingest Pipeline

6.1 Ingest Endpoints

Vantage accepts events through five paths. Every path ends at the same D1 INSERT logic, so any feature that reads the events table works the same regardless of which ingest path produced the row.

Endpoint	Client	Notes
POST /v1/events	SDKs, direct API, CLI, local proxy	One event per request. Idempotent on (event_id, org_id).
POST /v1/events/batch	SDKs when flushing buffered events	Up to 500 events per request. D1 batch API (single round-trip).
POST /v1/otel/v1/metrics	OTel collectors (Claude Code, Gemini CLI, any OTLP source)	OTLP format. Writes to otel_events and cross_platform_usage.
POST /v1/otel/v1/logs	OTel log-based metrics (Codex, Cline)	Same target tables as metrics path.
GitHub Copilot sync	Worker cron (Sundays UTC)	Polls Copilot Metrics API, upserts into cross_platform_usage.

6.2 Single Event Pipeline

```
POST /v1/events
  Body: EventIn (see types.ts)

1. authMiddleware (cookie or Bearer)
2. Viewer role block — 403 if role == 'viewer'
3. checkFreeTierLimit()
   -> SELECT COUNT(*) FROM events
      WHERE org_id=? AND created_at >= start_of_month
   -> If plan=='free' AND count+1 > 50,000:
      429 { error, events_used, events_limit, upgrade_url }
4. Field normalization (buildInsertStmt):
   -> Accept both SDK aliases and canonical names
   -> timestamp: event timestamp if provided, else Date.now()
5. INSERT OR IGNORE INTO events
   (idempotent on event_id+org_id)
6. broadcastEvent():
   KV.put("stream:{orgId}:latest", payload, TTL=60s)
7. maybeSendBudgetAlert()  (async, non-blocking)
8. Return 201 { ok: true, id: event_id }
```

6.3 Batch Event Pipeline

```
POST /v1/events/batch
  Body: {
    events: EventIn[],    // max 500
    sdk_version?: string,
    sdk_language?: string
  }

1-3. Same as single (plus batch size check)
4. Build array of D1PreparedStatement via buildInsertStmt()
5. c.env.DB.batch(stmts) — single D1 round-trip
6. broadcastEvent() for the last event in the batch
7. Return 201 { ok: true, accepted: N, failed: M }
```

Always prefer batch for SDK flushes

The SDK buffers events client-side and flushes in batches to reduce network overhead and stay under the rate limit. A batch of 500 costs one rate-limit token, not 500. The D1 batch API commits all statements atomically.

6.4 Free Tier Enforcement

The free tier allows 50,000 events per calendar month per org. Enforcement is a COUNT(*) query against a month-start epoch — SQLite's strftime('%s', 'now', 'start of month') avoids timezone ambiguity because all timestamps in the events table are UTC.

```
FREE_TIER_LIMIT = 50,000 events/month

SELECT COUNT(*) FROM events
WHERE org_id = ?
  AND created_at >= strftime('%s', 'now', 'start of month')

if plan == 'free' AND current_count + new_events > 50_000:
  return 429 {
    error: "Free tier limit reached",
    events_used: N,
    events_limit: 50000,
    upgrade_url: "https://vantageaiops.com/signup.html"
  }
```

6.5 Quality Scores (Async Writeback)

Quality scores are not computed at ingest time. The LLM-as-judge (Claude Opus 4.6) evaluates events offline and PATCHes them afterwards. The dashboard shows the average efficiency score over the period; if no scores exist yet, a default of 74 is shown to avoid empty KPI cards.

```
PATCH /v1/events/{event_id}/scores
{
  "hallucination_score": 0.12,   // 0-1 (lower = better)
  "faithfulness_score":  0.95,   // 0-1 (higher = better)
  "relevancy_score":     0.88,
  "consistency_score":   0.91,
  "toxicity_score":      0.01,
  "efficiency_score":    72.0    // 0-100 composite
}
-> UPDATE events SET ... WHERE id=? AND org_id=?
```

6.6 Semantic Cache and prompt_hash Dedup

When an SDK sends a prompt_hash (SHA-256 first 16 hex chars of the prompt), the worker checks KV for a recent identical prompt within the last 24 hours. If one is found, the event is marked cache_hit=1 and the response includes a cache_warning string that the SDK surfaces to the caller.

```
if body.prompt_hash:
  if not re.match(r'^[0-9a-f]{32,128}$', body.prompt_hash):
    return 400 "Invalid prompt_hash format"

  existing = KV.get("phash:{orgId}:{hash}")
  if existing:
    body.cache_hit = 1
    response.cache_warning =
      f"Duplicate detected — identical prompt sent {N}m ago. "
      f"Wasted: ${cost}"

  INSERT INTO events ... (with cache_hit value)

  # Write fresh entry even on hit, to extend the TTL
  KV.put("phash:{orgId}:{hash}",
         { event_id, cost_usd, model, ts },
         { expirationTtl: 86400 })
```

Next: [Chapter 7 — Analytics Engine] — how the dashboard turns raw events into KPI cards and charts.

Chapter 7 — Analytics Engine

7.1 Endpoint Map

All analytics endpoints share the teamScope() helper that appends AND e.team = ? when the authenticated member has a scope_team set. This is the data-isolation mechanism for scoped members — enforced in SQL, not in application code.

Endpoint	Time window	Primary aggregation
GET /v1/analytics/summary	today / 30-day MTD / last 30 min	cost, tokens, requests, budget%
GET /v1/analytics/kpis?period=N	N days (max 365)	totals + averages + streaming count
GET /v1/analytics/timeseries?period=N	N days	daily cost/tokens/requests
GET /v1/analytics/models?period=N	N days	per-model breakdown (top 25 by cost)
GET /v1/analytics/teams?period=N	N days	per-team cost + budget%
GET /v1/analytics/traces?period=N	N days (max 30)	agent trace summary (top 100)
GET /v1/analytics/cost?period=N	N days + today	CI cost gate (total + today)

7.2 Summary Calculation

The summary endpoint returns the values shown on the top row of the dashboard: today's cost, month-to-date cost, current session cost (a "now" feel), and the budget percentage consumed. It computes all four values with three SELECTs plus a budget lookup.

```
-- today_cost_usd / today_tokens / today_requests (last 24h)
SELECT SUM(cost_usd), SUM(total_tokens), COUNT(*)
FROM events WHERE org_id=?
  AND created_at >= (now - 86400)
  [AND team=?   -- if scope_team set]

-- mtd_cost_usd (last 30 days rolling — simpler than calendar)
SELECT SUM(cost_usd)
FROM events WHERE org_id=?
  AND created_at >= (now - 30*86400)

-- session_cost_usd (last 30 minutes — "now" feel)
SELECT SUM(cost_usd)
FROM events WHERE org_id=?
  AND created_at >= (now - 1800)
```

```
-- budget_pct
IF scope_team:
  SELECT budget_usd FROM team_budgets
  WHERE org_id=? AND team=?
ELSE:
  SELECT budget_usd FROM orgs WHERE id=?
pct = round((mtd_cost / budget) * 100)
```

7.3 Team Analytics with Budget Join

Teams analytics joins events and team_budgets so one query returns both spend and budget percentage per team. The alias on events.team is essential — without it, SQLite throws an ambiguous column error because both tables have a team column.

```
SELECT
  COALESCE(e.team, 'unassigned') AS team,
  SUM(e.cost_usd)                AS cost_usd,
  COALESCE(b.budget_usd, 0)      AS budget_usd,
  ROUND(
    SUM(e.cost_usd) / NULLIF(b.budget_usd, 0) * 100, 1
  ) AS budget_pct
FROM events e
LEFT JOIN team_budgets b
  ON b.org_id = e.org_id AND b.team = e.team
WHERE e.org_id = ?
  AND e.created_at >= ?
GROUP BY e.team
ORDER BY cost_usd DESC
```

7.4 Agent Tracing

Multi-agent workflows (LangChain, AutoGen, CrewAI) record a trace_id on every LLM call so all spans in a single agent run can be grouped. Spans where parent_event_id is NULL are root spans — the entry point of the agent workflow. span_depth tracks nesting level for tree reconstruction in the UI.

```
SELECT
  trace_id,
  MIN(agent_name)  AS name,       -- first agent name in trace
  COUNT(*)         AS spans,      -- LLM calls in the trace
  SUM(cost_usd)    AS cost,
  SUM(latency_ms)  AS latency,
  MAX(CASE WHEN parent_event_id IS NULL
```

```
           THEN 1 ELSE 0 END) AS has_root,
  MIN(created_at)  AS started_at
FROM events
WHERE org_id = ? AND trace_id IS NOT NULL
GROUP BY trace_id
ORDER BY started_at DESC
LIMIT 100
```

7.5 Cache Analytics

The KPIs endpoint surfaces cache savings and duplicate-call counts. Savings are computed in TypeScript after the D1 query because the savings rate varies per model (not expressible in a single SQL expression). The formula is: savings = (cached_tokens / 1e6) * (input_price - cache_read_price) per model.

```
-- KPIs query addition
COALESCE(SUM(cache_tokens), 0)   AS total_cache_tokens,
GROUP_CONCAT(model || ':' ||
             COALESCE(cache_tokens, 0)) AS cache_by_model,
COALESCE(SUM(CASE WHEN cache_hit=1 THEN 1 ELSE 0 END), 0)
                                 AS duplicate_calls,
COALESCE(SUM(CASE WHEN cache_hit=1 THEN cost_usd ELSE 0 END), 0)
                                 AS wasted_cost_usd

-- Response shape addition
{
  "cache_tokens_total":   1_420_000,
  "cache_savings_usd":    1.23,
  "cache_hit_rate_pct":   34.2,
  "duplicate_calls":      47,
  "wasted_cost_usd":      0.53
}
```

7.6 CI Cost Gate

The /v1/analytics/cost endpoint is designed to be called from a CI pipeline. It returns one number — today's AI cost — so the pipeline can fail if the cost crossed a threshold. This prevents runaway test suites and agent-in-a-loop incidents from accruing surprise invoices.

```
# In GitHub Actions:
- name: Check AI cost gate
  run: |
```

```
COST=$(curl -sf -H "Authorization: Bearer $VANTAGE_KEY" \
  "https://api.vantageaiops.com/v1/analytics/cost?period=1" \
  | jq '.today_cost_usd')
echo "Today's AI cost: $COST"
python -c "import sys; sys.exit(1 if float('$COST') > 5.0 else 0)"
```

Next: **Chapter 8 — Real-Time: Rate Limits, SSE, Alerts**

Chapter 8 — Real-Time: Rate Limits, SSE, Alerts

8.1 Rate Limiting

Vantage uses a fixed-window counter stored in Cloudflare KV. Each minute gets its own KV key; the key expires after 70 seconds (a 10-second buffer for clock skew). The limit is per-org (shared across all members of an org), not per-key.

```
async function checkRateLimit(
  kv: KVNamespace,
  orgId: string,
  limitRpm: number
): Promise<boolean> {
  const key = `rl:${orgId}:${Math.floor(Date.now() / 60_000)}`;
  const raw = await kv.get(key);
  const count = raw ? parseInt(raw, 10) : 0;
  if (count >= limitRpm) return false;
  // Fire-and-forget: non-blocking, best-effort
  kv.put(key, String(count + 1), { expirationTtl: 70 });
  return true;
}

// On 429 the response includes:
//   Retry-After: <seconds until next minute boundary>
//   X-RateLimit-Limit: 1000
//   X-RateLimit-Remaining: 0
```

Known tradeoff: boundary burst

Fixed window allows a client to send 1,000 requests at 11:59:59 and 1,000 more at 12:00:01 — effectively 2,000 in two seconds. For background telemetry this is acceptable. A sliding window implementation using Durable Objects is on the roadmap for customers with strict fairness requirements.

8.2 Real-Time Streaming (SSE)

Cloudflare Workers have a 30-second wall-clock limit. True persistent WebSocket connections require Durable Objects, which add cost and complexity. The current design is polling-over-SSE: a short-lived connection that polls KV for the latest event every 2 seconds and auto-reconnects when the worker tears down.

```
GET /v1/stream/:orgId?sse_token=XYZ

1. Validate sse_token (KV lookup, one-time use)
2. Open TransformStream (chunked HTTP response)
```

```
3. Background loop (max 25 seconds):
     while (Date.now() < deadline) {
       raw = KV.get("stream:{orgId}:latest")
       if (raw && raw.ts > lastTs) {
         sendEvent(raw)
         lastTs = raw.ts
       } else {
         sendPing()   // keep-alive heartbeat
       }
       await sleep(2000)
     }
4. writer.close() -> browser EventSource auto-reconnects
```

8.2.1 SSE Token Security

The browser EventSource API cannot attach an Authorization header. Two auth methods accommodate this:

- **?sse_token=** — 32-char hex token generated in GET /v1/auth/session, stored in KV with 120s TTL, one-time use (deleted on connect). Prevents replay from URL logs. This is what the dashboard uses.
- **?token=vnt_...** — legacy bearer token in query param. Accepted without KV lookup for SDK/direct callers, where browser-style log leakage is not a concern.

8.2.2 KV Broadcast Channel

When any event is ingested, the worker writes KV.put("stream:{orgId}:latest", payload, TTL=60s). The SSE loop polls this key every 2 seconds and sends the event if its timestamp is newer than the last sent timestamp.

> **Sampling is intentional**
>
> Only the latest event per org is buffered. High-frequency orgs (many events per second) see sampling, not every event. The live stream is a heartbeat view, not a complete event feed — use /analytics/timeseries for complete data.

8.3 Budget Alerts

Budget alerts fire at 80% and 100% of the monthly budget. The alert payload is a Slack Block Kit message, delivered via the org's configured Slack webhook. KV throttling prevents alert spam: one alert per type per hour.

```
async function maybeSendBudgetAlert(db, kv, orgId, mtdCost, budgetUsd) {
  const pct = (mtdCost / budgetUsd) * 100;
```

```
  let alertType = null;
  if (pct >= 100) alertType = 'budget_100';
  else if (pct >= 80) alertType = 'budget_80';
  if (!alertType) return;

  // Throttle: one alert per type per hour
  const throttleKey = `alert:${orgId}:${alertType}`;
  if (await kv.get(throttleKey)) return;

  // Slack webhook cached in KV (set when /v1/alerts/slack is called)
  const slackUrl = await kv.get(`slack:${orgId}`);
  if (!slackUrl) return;

  await sendSlackMessage(slackUrl, budgetAlertPayload);

  // Throttle for 1 hour
  await kv.put(throttleKey, '1', { expirationTtl: 3600 });
}
```

8.4 Anomaly Detection (Z-Score)

Beyond budget alerts, a cron-triggered worker runs every 10 minutes and computes a Z-score on the current hour's cost relative to the rolling 7-day hourly average. Anything above 3 sigma fires an anomaly alert. This catches runaway agents before they accrue a meaningful bill.

```
def is_anomaly(current_hour_cost, historical_costs):
    mean = statistics.mean(historical_costs)
    std  = statistics.stdev(historical_costs)
    z_score = (current_hour_cost - mean) / max(std, 0.001)
    return z_score > 3.0   # 3-sigma threshold
```

The enterprise pitch for this feature is a single sentence: "We caught a runaway agent before your invoice." That alone justifies annual renewal for most design partners.

Next: [Chapter 9 — Client Integration Patterns](#) — how each kind of caller reaches Vantage.

Chapter 9 — Client Integration Patterns

9.1 Six Client Archetypes

Vantage serves six distinct client types. Each has a different integration pattern, different auth needs, and different data characteristics. Understanding which archetype a user fits is essential for onboarding and support.

Type	Who	How they integrate
A	Python backend engineer	Drop-in OpenAIProxy / AnthropicProxy wrapper
B	JavaScript/Node.js engineer	Drop-in OpenAIProxy with streaming support
C	Multi-agent framework author	Direct POST /v1/events with trace_id
D	AI coding tool user (IDE)	MCP server reads analytics, SDK-less
E	Team admin / CTO	Dashboard (cookie session)
F	CI/CD pipeline	curl /v1/analytics/cost + exit code

9.2 Type A — Python Backend

The Python SDK wraps the OpenAI and Anthropic clients. The application code looks identical to vanilla OpenAI/Anthropic code — the only change is the import. The SDK silently POSTs one event per LLM call to api.vantageaiops.com in a background thread so the application is never blocked.

```
# pip install vantageaiops
from vantageaiops import OpenAIProxy

client = OpenAIProxy(api_key="vnt_myorg_...")
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "Hello"}]
)
# SDK silently posts event to api.vantageaiops.com/v1/events
# in a background thread — zero overhead on the main call.
```

Under the hood, the SDK:

8. Wraps the OpenAI/Anthropic client so existing code works unmodified.
9. Intercepts the response and reads the usage block.
10. Looks up pricing in the bundled MODEL_PRICES table.
11. Computes cost = (prompt_tokens/1e6 * input_price) + (completion_tokens/1e6 * output_price).
12. Queues an event in a background thread and POSTs to /v1/events.

9.3 Type B — TypeScript / Node.js

The JS SDK has the same API as the Python SDK plus streaming support. When the OpenAI streaming API is used, the SDK intercepts SSE chunks from OpenAI, counts tokens (or estimates from chunk count), and reports them after the stream closes.

```
// npm install vantageaiops
import { OpenAIProxy } from 'vantageaiops';

const client = new OpenAIProxy({
  apiKey: 'vnt_myorg_...',
  defaultTags: {
    team: 'frontend',
    feature: 'chat',
    user_id: userId,
  },
});

const response = await client.chat.completions.create({
  model: 'gpt-4o',
  stream: true,
  messages: [...],
});

for await (const chunk of response) {
  // forward to the user
}
// On stream close, SDK flushes one event with accurate token counts.
```

9.4 Type C — Multi-Agent Frameworks

LangGraph, AutoGen, CrewAI, and custom agent frameworks do not use the SDK directly. Each LLM call is a span in a larger workflow and needs a trace_id so all spans in a single agent run can be grouped in the Traces view. Integration is via direct POST /v1/events.

```
import requests, uuid

trace_id = str(uuid.uuid4())

# Root span
root_id = str(uuid.uuid4())
requests.post("https://api.vantageaiops.com/v1/events", json={
  "event_id":       root_id,
```

```
  "provider":       "anthropic",
  "model":          "claude-sonnet-4-6",
  "total_cost_usd": 0.0045,
  "total_tokens":   1200,
  "latency_ms":     2340,
  "trace_id":       trace_id,
  "parent_event_id": None,          # root span
  "agent_name":     "ResearchAgent",
  "span_depth":     0,
  "team":           "ai-platform",
}, headers={"Authorization": "Bearer vnt_..."})

# Child span
requests.post("https://api.vantageaiops.com/v1/events", json={
  "event_id":        str(uuid.uuid4()),
  "trace_id":        trace_id,
  "parent_event_id": root_id,
  "agent_name":      "SummarizerAgent",
  "span_depth":      1,
  # ...
})
```

Multi-agent budgets are a must-have

Multi-agent workflows can generate surprise invoices — an infinite loop at 2am is the canonical failure mode. Always set a per-team budget alert at 80% when you onboard a Type C user.

9.5 Type D — AI Coding Tool (MCP Client)

Developers using Claude Code, Cursor, or Windsurf run the vantageaiops-mcp server locally. Every MCP tool call is a JSON-RPC request that the MCP server forwards to the Vantage API. The tools surface cost data inline in the chat — "what did I spend on this project this week?" becomes a chat command.

```
VS Code / Cursor / Windsurf / Claude Code
    |
    +-- runs vantageaiops-mcp as a local process
            |
            +-- MCP tool call over stdio (JSON-RPC 2.0)
                  |
                  +-- HTTP GET /v1/analytics/* (Bearer auth)
                  +-- HTTP POST /v1/events
                        |
                        +-- Renders cost summary inline in chat
```

The twelve MCP tools: analyze_tokens, estimate_costs, get_summary, get_traces, get_model_breakdown, get_team_breakdown, get_kpis, get_recommendations, check_budget, compress_context, find_cheapest_model, optimize_prompt.

> **MCP is mostly a read path**
> Unlike other client types, MCP clients typically do not ingest events directly — AI coding tools do not expose their underlying LLM calls to user code. MCP is purely a read path for analytics of events that were ingested by the backing application or the OTel collector.

9.6 Type E — Team Admin (Dashboard)

The CTO, Head of Engineering, or DevOps lead uses the dashboard to monitor cost, set budgets, manage team members, and configure alerts. The dashboard is a static HTML/JS SPA served from Cloudflare Pages that polls the API with a session cookie.

Typical workflows:

- Morning dashboard review — check today_cost_usd, budget_pct, any anomalies
- Team management — invite a new engineer with scope_team so they only see their team's data
- Budget control — PUT /v1/admin/team-budgets/backend to set a $500/month cap
- Slack alert setup — Settings → Alerts → paste webhook URL
- Cost investigation — drill into Models view to see which model is driving cost

9.7 Type F — CI/CD Cost Gate

A CI pipeline fails the build if today's AI cost crossed a threshold. The client never looks at the dashboard. It is a binary gate.

```
# .github/workflows/test.yml
- name: AI cost gate
  run: |
    COST=$(curl -sf -H "Authorization: Bearer $VANTAGE_KEY" \
      "https://api.vantageaiops.com/v1/analytics/cost?period=1" \
      | jq '.today_cost_usd')
    echo "Today's AI cost: $COST"
    python -c "import sys; sys.exit(1 if float('$COST') > 5.0 else 0)"
```

Next: Chapter 10 — Cross-Platform Console & Adapters

Chapter 10 — Cross-Platform Console & Adapters

10.1 Why Cross-Platform

CTOs buy one thing first: "show me my total AI bill." Historically that required opening five dashboards — GitHub Copilot's seat report, the OpenAI usage page, the Anthropic console, Cursor's admin panel, and Datadog. Vantage consolidates all of them into one tab.

The cross-platform story also unlocks the per-developer question: "Which developer is driving our Copilot + Claude Code bill?" — something no provider can answer because each provider only sees their own slice. Vantage is structurally positioned to be the only answer.

10.2 Data Architecture

```
Ingestion paths
===============

OTel collectors (Claude Code, Gemini CLI, Cursor, Cline, ...)
  POST /v1/otel/v1/metrics
    -> otel_events (raw)
    -> cross_platform_usage (normalized per developer/period)

GitHub Copilot Metrics API (polled by Sunday cron)
    -> cross_platform_usage (tool_type='copilot_chat')

SDK events (OpenAI/Anthropic direct)
  POST /v1/events
    -> events table (used for the traditional /analytics/* views)

Read paths
==========

/v1/cross-platform/summary       whole-org totals
/v1/cross-platform/developers    per-developer table
/v1/cross-platform/trend         stacked daily cost chart
/v1/cross-platform/developer/:id modal detail
/v1/cross-platform/live          SSE last-50 feed
/v1/cross-platform/models        cost by model across tools
/v1/cross-platform/connections   provider status panel
/v1/cross-platform/budget        cross-tool budget check
```

10.3 The /trend Endpoint

The stacked daily cost chart is powered by one new endpoint, /v1/cross-platform/trend. The endpoint validates the period against an allowlist (7, 30, or 90 days only), builds a full calendar spine of N days,

and merges the SQL result into that spine so Chart.js always receives a continuous x-axis — even on days where a provider had zero cost.

```
const ALLOWED_DAYS = new Set([7, 30, 90]);
const rawDays = parseInt(c.req.query('days') ?? '30', 10);
if (!ALLOWED_DAYS.has(rawDays)) {
  return c.json({ error: 'days must be 7, 30, or 90' }, 400);
}

const rows = await db.prepare(`
  SELECT DATE(period_start) AS day,
         provider,
         SUM(cost_usd) AS cost
  FROM cross_platform_usage
  WHERE org_id = ?
    AND period_start >= ?
  GROUP BY day, provider
  ORDER BY day ASC
`).bind(orgId, cutoff).all();

// Build full calendar spine (never leave gaps in the x-axis)
const spine: string[] = [];
for (let i = rawDays - 1; i >= 0; i--) {
  const d = new Date();
  d.setUTCDate(d.getUTCDate() - i);
  spine.push(d.toISOString().slice(0, 10));
}

// Merge SQL rows into spine per provider
const providerSet = new Set(rows.map(r => r.provider));
const series = [...providerSet].sort().map(provider => ({
  provider,
  data: spine.map(day => {
    const row = rows.find(r => r.provider === provider && r.day === day);
    return row ? Number(row.cost) : 0.0;
  }),
}));

return c.json({
  period_days: rawDays,
  days: spine,
  providers: [...providerSet],
  series,
});
```

10.4 GitHub Copilot Adapter

The GitHub Copilot Metrics API went GA in February 2026. It is a REST API (not OTel) that returns per-developer seat usage, acceptance rates, and active user counts. Vantage is the first third-party tool to ship an adapter.

Property	Value
Auth	GitHub personal access token with manage_billing:copilot scope
Token storage	AES-256-GCM encrypted in KV (never in D1)
Sync schedule	Sundays 00:00 UTC (cron trigger)
Idempotency	ON CONFLICT upsert into cross_platform_usage
Cost math	$19/user/month * active_users per period
Endpoints	POST /v1/copilot/connect, DELETE /v1/copilot/connect, GET /v1/copilot/status

10.5 Datadog Exporter

Vantage pushes two gauge metrics to the customer's own Datadog: vantage.ai.cost_usd and vantage.ai.tokens. This meets customers where their monitoring already lives — every enterprise customer already pays for Datadog and already has dashboards set up. Vantage becomes a first-class data source in those dashboards without the customer having to build anything.

Property	Value
Auth	Datadog API key, AES-256-GCM encrypted in KV
Sites	datadoghq.com, datadoghq.eu, ddog-gov.com, us3, us5
Push schedule	Hourly cron
Idempotency	KV-guarded with 23h TTL per calendar day per org
Metrics	vantage.ai.cost_usd, vantage.ai.tokens (both gauges)
Tags	provider, model, developer_id, org_id

Datadog is not a competitor

Datadog charges $15+/host and is observability-focused. Their LLM observability product exists but is not cost-primary and does not track AI coding tools. Vantage integrates with Datadog rather than competing with it — "Datadog is for infra, Vantage is for AI budgets, and Vantage ships data into Datadog."

10.6 Anonymized Benchmarks

Every customer can opt in to contribute their metrics to anonymized cross-company benchmark snapshots. k-anonymity is enforced: a cohort with fewer than 5 contributing orgs returns 404. org_id

never appears in snapshot rows themselves — only in the benchmark_contributions join table — so an anonymized query can never be re-identified.

Dimension	Values
Size bands	1-10, 11-50, 51-200, 201-1000, 1000+ employees
Industries	SaaS, fintech, ecommerce, healthtech, AI-native, other
Cadence	Quarterly snapshots
Metrics	cost_per_dev_month, tokens_per_dev_month, cache_hit_rate per model
Percentiles	p25 / p50 / p75 / p90
Floor	k-anonymity: sample_size >= 5 or return 404
Endpoints	POST /v1/benchmark/contribute, GET /v1/benchmark/percentiles, GET /v1/benchmark/summary

---

Next: **Chapter 11 — Semantic Caching & Waste Detection**

Chapter 11 — Semantic Caching & Waste Detection

11.1 The Three Phases

Waste detection and caching ship in three phases, ordered by value-per-effort. Phase 1 fixed a broken KPI card and shipped real cache analytics in one day. Phase 2 adds SHA-256 exact-match dedup detection across the whole org in 2–3 days. Phase 3 — true semantic caching via Cloudflare Vectorize — is deferred until Phase 2 data validates the duplicate-call rate is high enough to justify the infrastructure.

Phase	Technique	Status	Value
1	Fix cache analytics (surface cache_tokens)	Shipped	Immediate — fixes broken KPI, zero friction
2	SHA-256 prompt_hash dedup + KV 24h window	Shipped	Proactive — agent gets real-time duplicate warning
3	Workers AI embeddings + Vectorize cosine match	Deferred	Requires separate check_cache pre-call tool; revisit after Phase 2 data

11.2 Phase 1 — Cache Analytics

Provider-native caching (Anthropic prompt caching, OpenAI cache reads) was already reaching Vantage via OTel: the cache_tokens column existed and was populated. But the dashboard "Cache Savings" KPI card showed zero always — the GET /v1/analytics/kpis endpoint never queried the column. Phase 1 fixed that in three places: the KPIs SQL query, a new shared lib/pricing.ts with estimateCacheSavings(), and the dashboard read.

```
// lib/pricing.ts — shared pricing table + helper
export function estimateCacheSavings(
  cacheByModel: Record<string, number>
): number {
  let total = 0;
  for (const [model, cachedTokens] of Object.entries(cacheByModel)) {
    const price = MODEL_PRICES[model] ?? MODEL_PRICES.default;
    // Savings per MTok: input price minus cache-read price.
    const perMTok = price.input - price.cache_read;
    total += (cachedTokens / 1_000_000) * perMTok;
  }
  return total;
}

// routes/analytics.ts — KPIs endpoint addition
const kpis = await db.prepare(`
  SELECT
```

```
    SUM(cost_usd)                      AS total_cost,
    SUM(total_tokens)                  AS total_tokens,
    COUNT(*)                           AS total_requests,
    COALESCE(SUM(cache_tokens), 0)     AS total_cache_tokens,
    GROUP_CONCAT(model || ':' ||
                 COALESCE(cache_tokens, 0)) AS cache_by_model
  FROM events WHERE org_id = ? AND created_at >= ?
`).bind(orgId, cutoff).first();

const cacheByModel = parseCacheByModel(kpis.cache_by_model);
const cacheSavingsUsd = estimateCacheSavings(cacheByModel);
const cacheHitRatePct = kpis.total_tokens
  ? (kpis.total_cache_tokens / kpis.total_tokens) * 100
  : 0;
```

11.3 Phase 2 — Prompt Hash Dedup

Phase 2 adds a prompt_hash column to the events table (first 16 hex chars of SHA-256) and a KV lookup on ingest. If the same hash has been seen in the last 24 hours, the event is marked cache_hit=1 and the response includes a warning the SDK surfaces to the user.

Migration 0007_prompt_hash.sql:

```
ALTER TABLE events ADD COLUMN prompt_hash TEXT;
ALTER TABLE events ADD COLUMN cache_hit   INTEGER NOT NULL DEFAULT 0;
CREATE INDEX idx_events_prompt_hash
  ON events(org_id, prompt_hash)
  WHERE prompt_hash IS NOT NULL;
```

Worker logic:

```
// routes/events.ts — additions
if (body.prompt_hash) {
  // Validate format: hex, 32-128 chars
  if (!/^[0-9a-f]{32,128}$/i.test(body.prompt_hash)) {
    return c.json({ error: 'Invalid prompt_hash format' }, 400);
  }

  const key = `phash:${orgId}:${body.prompt_hash}`;
  const existing = await kv.get(key, { type: 'json' });

  if (existing) {
    body.cache_hit = 1;
    response.cache_warning =
```

```
      `Duplicate detected — identical prompt sent ${minutesAgo}m ago. ` +
      `Wasted: $${existing.cost_usd}`;
  }

  await db.insertEvent(body);

  // Refresh TTL on every write — keeps hot prompts hot
  await kv.put(key, JSON.stringify({
    event_id: body.event_id,
    cost_usd: body.total_cost_usd,
    model:    body.model,
    ts:       Date.now(),
  }), { expirationTtl: 86400 });
}
```

🔧 FIXED v1.1: Phase 3 now has explicit decision gates. Ship if: (1) duplicate-call rate > 5% [measure in May 2026], (2) Helicone ships semantic cache, (3) customer requests it

11.4 Phase 3 — Semantic Cache (Deferred)

The deferred semantic cache uses Cloudflare Vectorize (the vector database) and Workers AI (BGE small English embeddings). The architectural challenge — and the reason it is Phase 3, not Phase 1 — is that track_llm_call is called after the LLM responds. It cannot return a cached response. True semantic caching requires a separate check_semantic_cache pre-call tool and a fundamental agent workflow change.

```
// Vantage semantic cache check (deferred Phase 3)
export async function checkSemanticCache(
  prompt: string,
  orgId: string,
  threshold = 0.92,
): Promise<CacheHit | null> {
  // 1. Embed the prompt with Workers AI
  const { data } = await workersAI.run(
    '@cf/baai/bge-small-en-v1.5',
    { text: [prompt] },
  );

  // 2. Query the org-scoped Vectorize namespace
  const results = await vectorize.query(data[0], {
    topK: 1,
    namespace: orgId,
    returnMetadata: 'all',
  });
```

```
  if (!results.matches.length) return null;
  if (results.matches[0].score < threshold) return null;

  // 3. Return cache hit + savings
  const best = results.matches[0];
  return {
    response:         best.metadata.response,
    saved_usd:        best.metadata.cost_usd,
    similarity_score: best.score,
  };
}
```

Ship this before Helicone does

Helicone is exact-match only right now. The semantic caching window is still open but closing. Phase 2 data will tell us whether the duplicate-call rate is above 5% — the threshold where Phase 3 is worth the Vectorize infrastructure spend.

Next: [Chapter 12 — Vantage Agent CLI]

Chapter 12 — Vantage Agent CLI

12.1 The Problem

Vantage Agent (npx vantageai-cli) is the command-line tool that sits between a user and any AI agent — Claude Code, OpenAI Codex CLI, Gemini CLI — and provides prompt optimization, cost tracking, anomaly detection, session persistence, and budget enforcement. The original implementation required paid Anthropic API credits to work, which locked out users with Claude Max subscriptions, Codex Pro, or Gemini free tier.

The identity is important:

> **Vantage Agent = middleware intelligence layer, not a wrapper**
>
> Integration model: prompt-in → optimize → send → observe output → track. The agent never intercepts tool calls. It is backend-agnostic and works with any CLI that accepts "echo prompt | agent" or "agent -p prompt".

12.2 Session-Centric Architecture

The central abstraction is VantageSession — a single serializable unit that owns the conversation history, an optional persistent subprocess, and a pluggable Backend adapter. Intelligence runs as pre/post hooks around session.send(). Backends self-declare their capabilities so the session picks the right execution path without scattered if backend == conditionals.

```
User Input
    |
    v
+-----------------------------------------------------+
|  VantageSession  (single serializable unit)        |
|                                                     |
|  PRE-SEND HOOKS  (pure, zero LLM tokens)           |
|    classifier -> optimizer -> budget_check         |
|         |            |            |                |
|                                                     |
|  ConversationHistory  (always present)             |
|    messages: [{role, text}]   backend-agnostic     |
|    session_id: UUID4                                |
|    working_dir, created_at, last_active_at          |
|         |                                           |
|  Backend  (capability-declared adapter)            |
|    name: "claude" | "codex" | "gemini"             |
|    capabilities = {                                 |
|      supports_process:  bool,                       |
```

```
|      supports_streaming: bool,                      |
|      token_count: "exact"|"estimated"|"free_tier", |
|      tool_format: "anthropic"|"openai"|"google",   |
|    }                                                |
|         |                                           |
|  HYBRID SEND PATH                                   |
|    if supports_process and process.alive:           |
|       stdin/pipe                                    |
|    else:                                            |
|       context-replay (inject history)               |
|         |                                           |
|  POST-SEND HOOKS                                    |
|    cost_tracker -> anomaly -> recommendations       |
|    -> telemetry (async, non-blocking)               |
+-----------------------------------------------------+
    |
    v
Output + Cost Report
```

12.3 Session Lifecycle

```
# Create
session = VantageSession.create(backend="claude", cwd="/project")

# Send (hybrid path chosen automatically by capability)
response = session.send("refactor this function")

# Persists after every successful turn to
#   ~/.vantage/sessions/<session_id>.json

# Resume after crash or in a new terminal
session = VantageSession.resume(session_id)

# List all sessions sorted by last_active_at
sessions = SessionStore.list()
```

Session file schema:

```
{
  "id": "uuid4",
  "backend": "claude",
  "cwd": "/project",
  "messages": [
    {"role": "user", "text": "..."},
    {"role": "assistant", "text": "..."}
```

```
  ],
  "cost_summary": {
    "total_cost_usd": 0.0234,
    "total_input_tokens": 1200,
    "total_output_tokens": 400,
    "backend": "claude",
    "token_count_confidence": "estimated"
  },
  "created_at":    "2026-04-08T10:00:00Z",
  "last_active_at": "2026-04-08T10:45:00Z"
}
```

12.4 Permission System (Claude CLI Backend)

The permission system is a two-layer design. Layer A ("tiered startup") runs on first launch: the user picks a tier (Read-only / Standard / Full / Custom) which sets session defaults. Layer B ("PreToolUse hook + Unix socket") fires per tool call for anything not auto-approved by the tier. Both layers read and write the same shared permission store at ~/.vantage-agent/permissions.json.

The tiered prompt shown on first run with Claude CLI backend:

```
  Vantage Agent — Tool Permissions

  Select what Claude is allowed to do:

  [1] Read-only    Read, Glob, Grep              (safe, no prompt)
  [2] Standard     + Edit, Write                 (file changes, ask once)
  [3] Full         + Bash (shell)                (Bash always asks per-call)
  [4] Custom       Choose tools individually

  > _
```

Bash is never in auto-approve

Even the "Full" tier still requires per-call hook approval for every Bash command. The tier only determines which file tools auto-approve. This is a hard constraint — enterprise clients need to see every shell command before it runs.

12.4.1 Hook Architecture

```
vantage-agent process
|-- Main thread: REPL input loop
|-- Thread A: subprocess stdout reader (stream-json parser)
|     +-- stdout_pause_event: threading.Event
|-- Thread B: Unix socket server at
```

```
|             /tmp/vantage-perm-<pid>.sock
|     +-- On connection: sets stdout_pause_event,
|           shows permission prompt,
|           collects user input,
|           writes to shared permission store,
|           sends decision back to hook,
|           clears stdout_pause_event
+-- Subprocess: claude -p <prompt>
      --output-format stream-json
      --verbose
      --permission-mode bypassPermissions
      --resume <session_id>
      --no-session-persistence
      --model <model>
      --settings /tmp/vantage-<pid>-settings.json
            +-- PreToolUse hook -> ~/.vantage-agent/perm-hook.sh
                 |-- Read always_approved -> fast-path exit 0
                 |-- Read always_denied   -> fast-path exit 2
                 +-- Connect to socket, prompt user, apply decision
```

12.4.2 Why bypassPermissions

The flag --permission-mode bypassPermissions disables Claude Code's own permission UI but (verified by live test) does not suppress PreToolUse hooks. This means Vantage's hook is the only permission gate — no duplication with Claude Code's own UI, and the /allow REPL command takes effect on the next call without restarting the subprocess.

The settings file is passed with --settings, which replaces (not merges) ~/.claude/settings.json. Vantage Agent reads the user's existing settings, merges the user's PreToolUse hooks with the Vantage hook, and writes a temporary merged settings file before spawning the subprocess.

12.5 Cost Display for Subscription Users

When the backend is claude CLI, the user is likely a Claude Max subscriber who paid once and gets unlimited usage. Vantage shows cost in a way that reinforces the value of the subscription rather than charging them for imaginary tokens:

```
↳ 1,204 tokens · API-equivalent: $0.0097
   [Max subscription: $0.00 actual]

Session summary:
  API-equivalent total: $0.42
  Actual paid:          $0.00
```

```
Your Max subscription saved you $0.42 this session.
```

Next: **Chapter 13 — MCP Server & Local Proxy**

Chapter 13 — MCP Server & Local Proxy

13.1 MCP Server Overview

The MCP (Model Context Protocol) server vantageaiops-mcp exposes twelve tools that work inside Claude Code, Cursor, Windsurf, and any MCP-compatible editor. Developers get cost data inline in their chat — "how much did I spend on this project today?" becomes a natural-language command.

Tool	Category	Purpose
analyze_tokens	Optimizer	Count tokens for a given prompt
estimate_costs	Optimizer	Estimate cost before sending
optimize_prompt	Optimizer	Compress prompt via LLMLingua + regex
compress_context	Optimizer	Summarize old messages in a long conversation
find_cheapest_model	Optimizer	Recommend the cheapest model for a task type
get_summary	Analytics	Today / MTD / session cost
get_kpis	Analytics	KPI card data including cache savings
get_model_breakdown	Analytics	Top 25 models by cost
get_team_breakdown	Analytics	Per-team cost + budget%
get_traces	Analytics	Top 100 agent traces
get_recommendations	Analytics	Savings recommendations
check_budget	Analytics	Budget percentage used

13.2 Optimizer vs Analytics Tools

Optimizer tools work offline — they do not need a Vantage API key. They run purely locally using the bundled pricing table and LLMLingua compressor. Analytics tools need a Vantage API key because they read from the events and otel_events tables via /v1/analytics/*.

> **Why optimizer tools are offline**
>
> Offline optimizer tools remove friction for first-time users. A developer can install vantageaiops-mcp, run optimize_prompt on a prompt they are drafting, and see the compressed output in seconds — no signup required. Once they see value, they sign up for the free tier and plug in a key to unlock analytics tools.

13.3 MCP Client Configuration

```
// .mcp.json in a repo, or ~/.config/claude-code/mcp.json
{
```

```
  "mcpServers": {
    "vantage": {
      "command": "npx",
      "args": ["-y", "vantageaiops-mcp"],
      "env": {
        "VANTAGE_API_KEY": "vnt_myorg_..."
      }
    }
  }
}
```

13.4 Local Proxy Gateway

Some organizations have strict data-residency rules: prompts must never leave the developer's machine. The Vantage Local Proxy is a local HTTP server that sits between the developer's IDE and the LLM provider, captures token/cost metadata, and sends only stats — never prompts — to the Vantage API.

```
+----------------+       +-------------------+       +--------------+
|  IDE / CLI     | ----> |  vantage-local-   | ----> | OpenAI /     |
|  (OpenAI SDK)  |       |  proxy (localhost) |       | Anthropic    |
+----------------+       +-------------------+       +--------------+
                                   |
                                   | stats only
                                   | (no prompts)
                                   v
                         +-------------------+
                         | api.vantageaiops  |
                         |  /v1/events       |
                         +-------------------+
```

13.5 Three Privacy Modes

Mode	Stats sent	Prompts sent	Metadata sent
strict	yes	no	no (no user_id, no project)
standard	yes	hashed only	hashed user_id, team, project
relaxed	yes	yes (optional)	full metadata + optional prompt text for debugging

The default is standard. Strict mode is positioned as the enterprise-procurement unlock — it is what lets Vantage get through security review at companies that would otherwise refuse any tool that sees prompts.

13.6 Session-Centric Integration Points

The MCP server, local proxy, and vantage-agent CLI are all independent Node/TypeScript or Python subsystems. The 2026-04-08 architecture audit found that session_id support was inconsistent across them — vantage-agent defined the model correctly, but the MCP and local proxy did not. Three fixes were identified:

Component	Gap	Fix
vantage-mcp	track_llm_call had no session_id field	Add optional session_id to input schema; pass through to /v1/events body
vantage-local-proxy	UUID generated per process; no resume	Add --resume <id> and --session-id <id> CLI flags, load from ~/.vantage/sessions/
vantage-worker OTel	session_id extracted but never rolled up	Add otel_sessions rollup table and GET /v1/sessions endpoint

🔧 FIXED v1.1: Added new Section 13.6.2 documenting GET /v1/sessions endpoint with full API spec (commit 3780bc7, PR #41)

13.6.1 otel_sessions Rollup

```
CREATE TABLE IF NOT EXISTS otel_sessions (
  org_id          TEXT    NOT NULL,
  session_id      TEXT    NOT NULL,
  provider        TEXT,
  developer_email TEXT,
  team            TEXT,
  model           TEXT,
  input_tokens    INTEGER NOT NULL DEFAULT 0,
  output_tokens   INTEGER NOT NULL DEFAULT 0,
  cached_tokens   INTEGER NOT NULL DEFAULT 0,
  cost_usd        REAL    NOT NULL DEFAULT 0,
  event_count     INTEGER NOT NULL DEFAULT 0,
  first_seen_at   TEXT    NOT NULL,
  last_seen_at    TEXT    NOT NULL,
  PRIMARY KEY (org_id, session_id)
);
CREATE INDEX idx_otel_sessions_org_last
  ON otel_sessions (org_id, last_seen_at DESC);
CREATE INDEX idx_otel_sessions_developer
  ON otel_sessions (org_id, developer_email);
```

Next: **Chapter 14 — Security Model & Audit Findings**

Chapter 14 — Security Model & Audit Findings

14.1 Security Principles

Vantage's security model is built on four principles that every engineer working on the codebase must understand.

13. API keys are never stored in plaintext. Only SHA-256 hashes in D1. Rotation is instant; there is no grace period.
14. Cross-org isolation is enforced at the query layer. Every query includes WHERE org_id = ?. There is no query in the entire worker that returns data across orgs.
15. Provider tokens (Copilot, Datadog) are AES-256-GCM encrypted in KV. They are never written to D1.
16. Sensitive operations go through audit log. Member invites, key rotations, config changes, and budget updates are all recorded in audit_events with actor + timestamp + payload.

14.2 API Key Security

Property	Implementation
Storage	SHA-256(rawKey) only; raw key shown once and never retrievable
Format encoding	vnt_{orgId}_{16-hex-random} — orgId for fast routing, 128-bit random for entropy
Hashing	Standard SHA-256 (libsodium-free, Web Crypto API)
Rotation	Instant, no grace period (unless you implement one)
Recovery	Single-use KV token, POST-only consume, 1-hour TTL

14.3 Frontend Security (app.html)

Concern	Mitigation
Chart.js CDN compromise	SRI hash enforced: integrity="sha384-..." + crossorigin="anonymous"
apiFetch exposure	Not on window; external scripts use one-time window.__cpRegister callback that self-nulls after cp-console.js claims it
api_base override	localStorage.api_base validated against explicit allowlist (api.vantageaiops.com, localhost)
XSS	All dynamic DOM writes use textContent; no innerHTML anywhere in app.html or cp-console.js
CSP	Content-Security-Policy in _headers for /app.html; connect-src restricts exfiltration targets
Session cookie	HttpOnly; SameSite=Lax; Secure; Domain=vantageaiops.com; Max-Age=2592000

14.4 Content Security Policy

```
# vantage-final-v4/_headers

/*
  X-Frame-Options: DENY
  X-Content-Type-Options: nosniff
  Referrer-Policy: strict-origin-when-cross-origin
  Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
  Cross-Origin-Opener-Policy: same-origin
  Cross-Origin-Resource-Policy: same-origin
  Permissions-Policy: camera=(), microphone=(), geolocation=()

/app.html
  Content-Security-Policy:
    default-src 'self' 'unsafe-inline' [trusted CDNs];
    connect-src 'self' https://api.vantageaiops.com
                       wss://api.vantageaiops.com;
    frame-ancestors 'none';
  Cache-Control: no-store, max-age=0

/app
  # (same as /app.html)

/superadmin.html
  # (same CSP with frame-ancestors 'none')
```

unsafe-inline is a known weakness

Removing unsafe-inline requires moving all inline <style> and <script> blocks to external files. Mitigated today by the strict connect-src (scripts cannot exfiltrate to unknown origins) plus frame-ancestors none. Full nonce-based CSP is on the hardening roadmap.

14.5 Security Audit Findings (2026-04-09)

A full audit was performed as part of the website enterprise overhaul. Nine findings were fixed in the same session; two were deferred with explicit rationale.

ID	Area	Severity	Status
FIND-001	HSTS header missing	Medium	Fixed
FIND-002	COOP/CORP headers missing	Low	Fixed
FIND-003	CSP missing frame-ancestors	Medium	Fixed
FIND-004	No brute-force protection on /v1/auth/session	High	Fixed

ID	Area	Severity	Status
FIND-005	prompt_hash accepts arbitrary strings	Low	Fixed
FIND-006	Competitive intel exposed in public docs	Low	Fixed
FIND-007	False SOC2/HIPAA/GDPR badges on landing page	High	Fixed
FIND-008	Unshipped features advertised as shipped	High	Fixed
FIND-009	Fake testimonials with named people and dollars	Medium	Fixed
DEFER-001	localStorage analytics cache in app.html	Medium	Deferred
DEFER-002	CSP unsafe-inline in app.html and docs.html	Low	Deferred

14.5.1 FIND-004 — Brute-Force Protection Detail

The /v1/auth/session endpoint had no rate limit. An attacker could attempt unlimited API key guesses. The /v1/auth/signup and /v1/auth/recover endpoints already had rate limiting; this one was missed. The fix added KV-backed IP rate limiting: 10 attempts per IP per 5-minute window, HTTP 429 with Retry-After: 300 on breach, graceful degradation if KV is unavailable.

🔧 FIXED v1.1: FIND-007 (false badges) and FIND-009 (fake testimonials) marked RESOLVED with timestamps - commit 9d85fba, a75c7e9 (2026-04-09)

14.5.2 FIND-007 — False Compliance Claims

The landing page displayed SOC 2 Ready, GDPR Compliant, and HIPAA Ready trust badges. None of these certifications are held. This is a false claim with legal risk and damages trust when enterprise procurement scrutinizes the claim. The fix replaced the badge grid with a list of actually implemented controls: SHA-256 hashing, HTTP-only sessions, audit log, TLS, rate limiting, and the local proxy. No SOC 2 / HIPAA / GDPR badges will return until those certifications are held.

14.5.3 DEFER-001 — localStorage Cache

app.html caches analytics API responses in localStorage under a key prefixed with the org_id. Session authentication itself uses HTTP-only cookies (secure), but localStorage is accessible to any JS on the page, including XSS payloads. The current caching stores analytics response data, not the raw API key — the risk is data leakage, not credential theft. Removing the caching would require a meaningful refactor. The session cookie model is already XSS-resistant, so this is tracked for a future hardening sprint.

Next: [Chapter 15 — CI/CD, Testing & Operational Runbook]

Chapter 15 — CI/CD, Testing & Operational Runbook

15.1 Branch Strategy

```
main (production)
 +-- v1.0, v1.1 (version branches — working branches)
 |     +-- v1.0/P001-fix-nav (feature branches)
 +-- backup/v1.0 (immutable snapshot, never modified)
```

Rule: Cloudflare always deploys from main. Version branches are a testing layer. Feature branches merge → version branch → all tests pass → auto-create backup → manual PR approval to main → Cloudflare deploys.

15.2 Workflow Overview

Workflow	Trigger	Action
deploy.yml	Push to main (frontend changes)	Wrangler Pages deploy with 4-attempt retry
deploy-worker.yml	Push to main (worker changes)	TypeScript check then Wrangler worker deploy
ci-version.yml	Push to v[0-9]* branches	Full test suite, backup branch, open PR to main
ci-feature.yml	Push to v*/P* feature branches	Fast smoke tests only
ci-test.yml	Post-deploy on main	Full test validation against live production
repo-backup.yml	Push to main + daily 03:00 UTC	Mirror to Amanjain98/Vantage-AI + ZIP artifact
setup-branch-protection.yml	Manual dispatch	Apply GitHub branch protection rules

15.3 Active Test Suites

Vantage has 41 test suites and 283+ individual checks. All tests hit the live API — no mocking. This is a deliberate choice: it makes tests slower but catches real regressions in the whole pipeline (worker + D1 + KV + Pages).

Suite	Directory	Coverage
32	32_audit_log	Audit event creation, pagination, role guard
33	33_frontend_contract	API shape contract checks for all frontend-consumed endpoints
34	34_vega_chatbot	Chatbot / recommendations widget API
35	35_cross_platform_console	Cross-platform console (all 8 endpoints)
35b	35_recommendations	Recommendations engine API

Suite	Directory	Coverage
36	36_semantic_cache	Semantic cache hit/miss + prompt_hash dedup
37	37_all_dashboard_cards	All dashboard card data shapes (9 classes, ~90 tests)
38	38_security_hardening	OWASP top-10 guards, injection, CORS
39	39_copilot_adapter	Copilot connect / disconnect / sync flow
40	40_benchmark	Benchmark contribute, percentiles, k-anonymity floor
41	41_datadog_exporter	Datadog connect, encrypt / decrypt, sync

15.4 Suite 37 — The Regression Guard

Suite 37 is the most important one for preventing silent data-model regressions. It seeds known data from all five ingestion paths (OTel, JS SDK, MCP tool, local-proxy, direct API) and then asserts every dashboard card shows the correct combined totals, fields, and cross-source aggregation. A regression like "timeseries queries the wrong table" — the exact bug the suite was built to catch — cannot slip past it.

Nine test classes, roughly ninety tests:

Section	Class	Cards Covered
A	TestOverviewKPICards	Total spend, costliest tool, active devs, budget, token usage, cache savings, daily spend chart, tool comparison, live feed
B	TestSpendAnalysisCards	Period spend, top model, total requests, avg cost, spend trend, model table
C	TestTodayHourlyChart	Hourly spend chart
D	TestCostByModelTable	Cost by model (cross_platform_usage backed)
E	TestCostByTeam	Team pie chart + per-card team data
F	TestDeveloperCards	Developer list and drill-down
G	TestCrossPlatformSource	by_source breakdown, connections, budget
H	TestAdminAuditCards	Admin overview, audit log
I	TestConsistencyAssertions	Cross-endpoint total consistency, regression guards

15.5 Operational Runbook — Common Tasks

15.5.1 Deploy

```
# Deploy frontend (Pages)
cd vantage-final-v4
```

```
npx wrangler pages deploy .

# Deploy worker
cd vantage-worker
npx wrangler deploy

# Both in CI: handled by deploy.yml and deploy-worker.yml
# (automatic on push to main, 4-attempt exponential backoff on 504)
```

15.5.2 D1 Operations

```
# Run SQL directly on production D1
npx wrangler d1 execute vantageai-production \
  --command "SELECT COUNT(*) FROM events"

# Apply a migration
npx wrangler d1 migrations apply vantageai-production --remote

# Backup (export to local file)
npx wrangler d1 export vantageai-production --output backup.sql

# Check table sizes
npx wrangler d1 execute vantageai-production --command \
  "SELECT name, SUM(pgsize) AS bytes FROM dbstat
   GROUP BY name ORDER BY bytes DESC"
```

15.5.3 KV Operations (Unblock a Stuck Org)

```
# List all keys with a prefix
npx wrangler kv:key list --namespace-id=$KV_ID --prefix="rl:org_abc"

# Get a value
npx wrangler kv:key get --namespace-id=$KV_ID "slack:org_abc"

# Delete a stuck rate limit key (unblock an org immediately)
npx wrangler kv:key delete --namespace-id=$KV_ID \
  "rl:org_abc:123456789"

# Delete a stuck alert throttle (re-enable alerts for that type)
npx wrangler kv:key delete --namespace-id=$KV_ID \
  "alert:org_abc:budget_80"
```

15.5.4 D1 Schema Migration Pattern

```
# Create migration file
npx wrangler d1 migrations create vantageai-production \
  add_prompt_hash

# Edit the generated file at
#   vantage-worker/migrations/0007_add_prompt_hash.sql

# Apply to production
npx wrangler d1 migrations apply vantageai-production --remote

# Never alter an applied migration after apply — create a
# new migration instead.
```

SQLite date format convention

All events and otel_events timestamps are INTEGER Unix epoch seconds. cross_platform_usage is the one exception: it uses TEXT "YYYY-MM-DD HH:MM:SS" because OTel semantic conventions send timestamps that way. Always use strftime('%s', ...) or unixepoch() — never mix TEXT and INTEGER in the same comparison.

15.6 Common Incidents

Symptom	Likely cause	First step
Dashboard empty	SSE token expired	Check browser devtools → Network → /v1/stream/* for 401
429 spam from one org	Rate limit counter stuck	wrangler kv:key delete rl:{orgId}:{minute}
Budget alert not firing	Slack webhook cache missing	Re-call POST /v1/alerts/slack/:orgId to refresh
Copilot sync missing data	GitHub token scope too narrow	Verify manage_billing:copilot scope on the PAT
Datadog metrics stopped	Site misconfigured	GET /v1/datadog/status; check error field
Dashboard cards show 0	Scoped member without events	Verify scope_team matches events.team column values

---

Next: Chapter 16 — Competitive Landscape & Moat

Chapter 16 — Competitive Landscape & Moat

16.1 The Category

Vantage is in the LLMOps and AI observability category as measured by SAM, but is positioned as "AI Coding FinOps" for marketing purposes — a wedge category that Vantage is deliberately claiming. The LLMOps SAM is $2–4B by 2027. The AI coding tool spend SAM is $8–12B by 2027. The SOM is ~$500M (roughly 10K companies × $50K ACV) realistic by Year 3–5.

16.2 Competitor Matrix (April 2026)

Dimension	Vantage	Helicone	LangSmith	Langfuse	Datadog LLM	Copilot Analytics
Free tier	50K events/mo	10K/mo	5K traces	50K units/mo	None	With seat
Paid entry	TBD	$20/seat	$39/seat	$29 flat	~$120/day	$10-19/user
OSS / self-host	No	Yes (Apache)	No	Yes (MIT)	No	No
AI coding tool tracking	Yes (OTel)	No	No	No	Partial	Own tool only
Copilot billing adapter	Yes	No	No	No	No	Native only
Per-dev cross-tool attribution	Yes	No	No	No	No	Own tool only
No proxy required	Yes	No	No	No	n/a	n/a
MCP server	Yes (12)	No	No	No	Yes (other)	No
CLI wrapper	Yes	No	No	No	No	No
Privacy / strict mode	Yes (3 modes)	No	No	No	No	No
Datadog exporter	Yes	No	No	No	n/a	No
Benchmark data (anonymized)	Yes (k-anon)	No	No	No	No	No
Audit log	Yes	Limited	Yes	Yes	Yes	No
Agent trace viz	Partial	Yes	Yes	Yes (GA)	Yes	No
Eval framework	No	Limited	Yes	Yes	No	No

16.3 Structural Weaknesses and Exploits

Competitor	Structural weakness	Vantage exploit
Helicone	Exact-match cache only. No AI coding tool OTel. No Copilot billing. No per-dev cross-tool attribution. Proxy = traffic interception risk for enterprise.	AI Spend Console + Copilot adapter + no-proxy + privacy modes beats on all enterprise dimensions.
LangSmith	Deep LangChain coupling. Tracing-heavy, not cost-primary. No multi-tool procurement story. No Copilot.	Cost-first narrative. Non-LangChain teams are underserved. CTOs buying Copilot do not care about LangChain.
Langfuse	MIT OSS is a moat. Strong eval + prompt mgmt. No AI coding tool OTel. No CLI wrapper. No Copilot.	"Langfuse shows your LLM calls. Vantage shows your AI coding bill." Different buyer: CTO vs ML engineer.
Datadog LLM	$15+/host explodes at scale. Observability focus, not cost intelligence. No AI coding tools. No Copilot attribution.	Purpose-built for AI spend. Cheaper. "Datadog is for infra, Vantage is for AI budgets." Vantage pushes data TO Datadog; does not compete with it.
OpenAI Dashboard	Shows only OpenAI. Provider-biased. No cross-model comparison.	Multi-provider neutrality. "Would you let your bank audit itself?"
Anthropic Console	Same — single provider. No Copilot, no Cursor, no competitive intelligence.	Vantage shows the full picture. Providers show only their slice.
GitHub Copilot Analytics	Copilot only. REST API (not OTel). No Cursor / Claude Code.	Vantage consumes Copilot Metrics API, normalizes it, shows it alongside everything else. Their data feeds Vantage's moat.
Palma.ai	Direct ICP competitor. Pre-PMF, limited marketing.	Ship first, own the "AI Coding FinOps" category.
CloudZero / Apptio	Cloud cost focus. Not built for LLM token economics. Slow to adapt.	AI-native from day one. Five-year acquisition targets, not near-term competition.

13A.1 Overview

The Claude Intelligence Plugin is a specialized MCP integration that makes LLM cost intelligence available directly within Claude Desktop, Cursor, VS Code, and other Claude-compatible IDEs. Exposes 12 cost-intelligence tools (analyze_tokens, estimate_costs, get_summary, check_budget, optimize_prompt, find_cheapest_model, estimate_costs, get_kpis, get_model_breakdown, get_team_breakdown, get_traces, compress_context) that work entirely offline using cached model pricing tables for 24 LLM models.

13A.2 Key Capabilities

- CI/CD cost gate integration for GitHub Actions workflows
- Budget tracking with optional API key integration
- Cost-aware model routing: suggest cheaper models with comparable quality
- Automatic prompt compression with preservation of code blocks and structured data

- Zero API latency — all inference happens locally in-process
- Works in Claude Desktop, Cursor, Windsurf, VS Code Claude extension, Cline
- Per-model cost comparison with savings calculation
- Prompt optimization recommendations (LLMLingua-based compression)
- Natural language cost queries: 'What's the cheapest Claude model for my use case?'
- Real-time cost estimation across all supported LLM providers
- Offline token analysis using local pricing tables (24 models: Claude, GPT-4o, o1, o3-mini, Gemini, Mistral, etc.)

16.4 Why Vantage Is the Only Full-Stack AI Coding Spend Platform

As of April 2026, Vantage is the only platform that covers the complete AI coding spend stack in one dashboard:

17. IDE tools (GitHub Copilot) — via Copilot Metrics API adapter, per-developer, per-seat cost.
18. LLM APIs (OpenAI, Anthropic, Google, Mistral) — via SDK + OTel collector, per-call tracking.

16A.1 The Four Permanent Advantages

VantageAI's defensibility is structural, not feature-based. Four permanent advantages providers cannot replicate:

1. PROVIDER CONFLICT LOCK-OUT: OpenAI cannot tell you Anthropic is cheaper. Anthropic cannot tell you Google outperforms on your workload. This conflict is permanent. CloudHealth survived AWS for 10+ years on this principle.

2. DATA NETWORK EFFECT: Company #500 makes benchmark data 10x more valuable for companies #1-499. The larger the network, the more accurate the pricing intelligence and negotiation leverage. Compounds forever.

3. TRUSTED THIRD PARTY: Auditors, boards, and regulators require an independent system of record. They will not accept a vendor dashboard. VantageAI is building the governance layer.

4. REGULATORY INEVITABILITY: EU AI Act, SOX-equivalent AI audit requirements, HIPAA AI governance—all coming. Compliance tools are 20-year contracts, not monthly subscriptions.

16A.2 Competitive Comparison (April 2026)

- vs OpenAI/Anthropic dashboards: Single-provider only, inherent bias. VantageAI=multi-provider neutrality. 'Would you let your bank audit itself?'

16A.3 Full-Stack AI Coding Spend Coverage

VantageAI is the ONLY platform covering the complete AI coding spend stack in one dashboard:

✓ IDE Tools (GitHub Copilot) — Copilot Metrics API adapter, per-developer per-seat cost
✓ LLM APIs (OpenAI, Anthropic, Google, Mistral, etc.) — SDK+OTel collector, per-call tracking
✓ Agent Frameworks (LangChain, AutoGen, any OTel-compatible) — OTLP endpoint, trace-level

attribution
✓ Existing Monitoring (Datadog) — exporter push, meets customers where they are

No competitor covers all four layers. This is the cross-stack narrative for every sales conversation.
- vs Datadog LLM: $15+/host explodes at scale. Observability focus, not cost intelligence. VantageAI pushes data TO Datadog—we don't compete.
- vs palma.ai: palma=agent governance+RBAC (CISO buyer). VantageAI=cost intelligence+ROI (CFO/VP Eng buyer). Complementary.
- vs GitHub Copilot Analytics: Limited to Copilot only. No cross-tool spend, no multi-model optimization, no org governance.
- vs Langfuse: Langfuse=strong on MIT OSS+eval+prompt mgmt. But no AI coding tool OTel, no CLI wrapper, no Copilot billing.
- vs LangSmith: LangSmith=deep LangChain coupling+eval-heavy. Non-LangChain teams underserved. No Copilot adapter, no AI coding tool OTel.
- vs Helicone: Helicone requires proxy deployment (enterprise risk). VantageAI: no-proxy via SDK+OTel+Local Proxy. Helicone=exact-match cache only; VantageAI building semantic cache.

19. Agent frameworks (LangChain, AutoGen, any OTel-compatible) — via OTLP endpoint, trace-level attribution.
20. Existing monitoring (Datadog) — via exporter push, meeting customers where they already live.

No competitor covers all four. This is the cross-stack narrative that should lead every sales conversation.

16.5 Watch List

Signal	Why it matters	What to do
Palma.ai pricing page	Direct ICP competitor	Check monthly; any raise accelerates category-claiming content
helicone/helicone GitHub	Semantic cache ship = less time	Watch for "semantic" in commit history
github/copilot-cli #2471	Copilot OTel parity	When shipped, update OTel adapter to consume natively
Langfuse Agent Graph	Product parity benchmark	Benchmark monthly for feature decisions
Copilot seat pricing	$19/user is embedded in cost model	Recompute Copilot adapter if price changes

---

Next: [Appendix — Glossary, Formulas, API Reference]

9A.1 Agent Trace Architecture

VantageAI's agent tracing enables cost attribution at span level for multi-step workflows:

- trace_id: Unique identifier for entire agent run (ResearchAgent→SearchTool→SummarizerAgent)
- parent_event_id: Identifies parent span in call tree (enables DAG reconstruction)
- span_depth: Nesting level (0=root, 1=first-level child, etc.)
- agent_name: Which agent made the call (ResearchAgent, code-review-agent, etc.)
- Cost per span: Individual LLM call cost
- Total workflow cost: Sum of all spans in trace_id

Enables CTOs to answer: 'That agent workflow cost $2.47—was that worth it?'

9A.2 Test Suite 37: Multi-Agent Testing (73 checks)

SPAN HIERARCHY (15): trace_id consistency, parent_event_id DAG validation, span_depth accuracy, cost accumulation
AGENT ROUTING (18): Agent-to-agent calls, tool calls, human-in-the-loop pauses, retry behavior
COST ATTRIBUTION (20): Cost per agent/workflow/model, tool call costs, cost comparisons
PERFORMANCE (10): Span latency (<5s), total workflow latency, parallelization detection
QUALITY (10): Hallucination score per agent, cost vs quality trade-off

All 73 tests pass on production. See Suite 37 dashboard for live results.

Appendix — Glossary, Formulas & API Reference

A.1 Glossary

Term	Meaning
ACV	Annual Contract Value — the total yearly revenue from one customer contract
BGE	Baidu General Embedding — the small English embedding model used by the semantic cache (bge-small-en-v1.5)
cross_platform_usage	D1 table that normalizes multi-tool spend per developer per period
D1	Cloudflare's serverless SQLite database
EventIn	The canonical input schema for POST /v1/events (see types.ts)
k-anonymity	Privacy constraint requiring that any released data represents at least k individuals. Vantage uses k=5.
KV	Cloudflare Workers KV — a global key-value store used for rate limits, SSE broadcast, alerts, and encrypted secrets
LLMLingua	Microsoft research project for compressing LLM prompts while preserving task performance
MCP	Model Context Protocol — the protocol Claude Code, Cursor, and Windsurf use to talk to tool servers over stdio
MRR	Monthly Recurring Revenue
MTD	Month-to-Date
MTok	Million Tokens — the unit provider pricing is quoted in
OTel / OTLP	OpenTelemetry / OpenTelemetry Protocol — the vendor-neutral metrics and logs protocol used for AI coding tool ingestion
Resend	Transactional email provider used for member invites and recovery emails
RPM	Requests Per Minute (rate limit unit)
SAM	Serviceable Addressable Market
session_id	UUID4 identifying a continuous conversation — propagated through vantage-agent, local-proxy, MCP, and OTel
SOM	Serviceable Obtainable Market
SSE	Server-Sent Events — the HTTP streaming protocol used for the live cost feed
TAM	Total Addressable Market
trace_id	UUID identifying one multi-agent workflow; all spans in the trace share it
VIE	Vantage Intelligence Engine — the 9-stage pipeline every event flows through

A.2 Key Formulas

A.2.1 Cost Estimation

```
cost_usd = (prompt_tokens    / 1e6) * price.input
         + (completion_tokens / 1e6) * price.output
         + (cache_tokens      / 1e6) * price.cache_read

# where price is MODEL_PRICES[canonical_model_name]
# and canonical names fold variants: claude-sonnet-4-6-20260301
# -> claude-sonnet-4-6
```

A.2.2 Cache Savings

```
savings_usd = sum over models of:
  (cached_tokens_for_model / 1e6) *
  (price.input - price.cache_read)

# Computed in TypeScript after the D1 query because the rate
# varies per model (not expressible in a single SQL expression).
```

A.2.3 Anomaly Z-Score

```
z_score = (current_hour_cost - mean_historical_hourly_cost)
          / max(stddev_historical_hourly_cost, 0.001)

# Alert if z_score > 3.0 (3-sigma threshold).
# Historical window: rolling 7 days of hourly cost.
```

A.2.4 Efficiency Score

```
efficiency_score = information_density
                 * task_completion_rate
                 * cost_ratio

where:
  information_density  = completion_tokens / prompt_tokens
  task_completion_rate = faithfulness_score     (from LLM judge)
  cost_ratio           = 1 - (actual_cost / max_expected_cost)

Result is normalized 0-100.
```

A.3 HTTP API Quick Reference

A.3.1 Auth

Endpoint	Method	Purpose
/v1/auth/session	POST	Exchange Bearer key for session cookie + sse_token
/v1/auth/signup	POST	Create a new org (rate limited)
/v1/auth/recover	POST	Send recovery email (always returns 200)
/v1/auth/recover/redeem	GET	Validate recovery token (peek only, does not consume)
/v1/auth/recover/redeem	POST	Consume recovery token, rotate key, return new key
/rotate	POST	Owner-only: rotate the root API key

A.3.2 Events

Endpoint	Method	Purpose
/v1/events	POST	Ingest one event (idempotent on event_id+org_id)
/v1/events/batch	POST	Ingest up to 500 events in one round-trip
/v1/events/:id/scores	PATCH	Async writeback of quality scores from LLM judge
/v1/otel/v1/metrics	POST	OTLP metrics ingest (AI coding tools)
/v1/otel/v1/logs	POST	OTLP logs ingest (Codex, Cline)

2A.1 Quick Install

PYTHON SDK:

```
$ pip install vantageaiops
$ export VANTAGE_API_KEY=vnt_yourorg_abc123
```

JAVASCRIPT SDK:

```
$ npm install vantageaiops
```

MCP SERVER (Claude Desktop, Cursor, Windsurf):

```
$ npx vantageaiops-mcp setup
```

CLI AGENT:

```
$ npx vantageai-cli
```

LOCAL PROXY (Privacy Mode):

```
$ npx vantage-local-proxy --api-key vnt_... --privacy strict
```

2A.2 Build from Source

PYTHON SDK:
```
$ git clone https://github.com/Kamal-Soft-Pvt-Ltd/vantage-python-sdk.git
$ cd vantage-python-sdk && pip install -e . && python -m pytest tests/
```

JAVASCRIPT SDK:
```
$ git clone https://github.com/Kamal-Soft-Pvt-Ltd/vantage-js-sdk.git
$ cd vantage-js-sdk && npm install && npm run build && npm test
```

MCP SERVER:
```
$ git clone https://github.com/Kamal-Soft-Pvt-Ltd/vantage-mcp.git
$ cd vantage-mcp && npm install && npm run build && npm link
```

CLI AGENT:
```
$ git clone https://github.com/Kamal-Soft-Pvt-Ltd/vantageai-agent.git
$ cd vantageai-agent && pip install -e . && python -m pytest tests/
```

2A.3 Upgrade & Uninstall

UPGRADE: pip install --upgrade vantageaiops OR npm update vantageaiops
UNINSTALL: pip uninstall vantageaiops OR npm uninstall vantageaiops
CLI AGENT: $ pip install --upgrade vantageai-cli / pip uninstall vantageai-cli
MCP: npx vantageaiops-mcp uninstall (or edit ~/.mcp.json manually)

A.3.3 Analytics

Endpoint	Method	Purpose
/v1/analytics/summary	GET	Today / MTD / session cost + budget%
/v1/analytics/kpis	GET	KPI card values incl. cache savings
/v1/analytics/timeseries	GET	Daily cost/tokens/requests
/v1/analytics/models	GET	Top 25 models by cost
/v1/analytics/teams	GET	Per-team cost + budget%
/v1/analytics/traces	GET	Top 100 agent traces
/v1/analytics/cost	GET	CI cost gate (total + today)

A.3.4 Cross-Platform

Endpoint	Method	Purpose
/v1/cross-platform/summary	GET	Org-wide cross-tool totals
/v1/cross-platform/developers	GET	Per-developer table
/v1/cross-platform/trend	GET	Stacked daily cost chart (period=7/30/90)
/v1/cross-platform/developer/:id	GET	Per-developer drill-down (admin, or self)

Endpoint	Method	Purpose
/v1/cross-platform/live	GET	SSE last-50 feed (email redaction for non-admin)
/v1/cross-platform/models	GET	Cost by model across tools
/v1/cross-platform/connections	GET	Provider status panel
/v1/cross-platform/budget	GET	Cross-tool budget check

A.3.5 Admin

Endpoint	Method	Purpose
/v1/admin/overview	GET	Admin summary (usage, members, recent audit events)
/v1/admin/members	GET	List members
/v1/admin/members	POST	Invite a new member (emails the raw API key once)
/v1/admin/members/:id/rotate	POST	Rotate a member key
/v1/admin/members/:id	DELETE	Remove a member
/v1/admin/team-budgets	GET	List team budgets
/v1/admin/team-budgets/:team	PUT	Upsert a team budget
/v1/alerts/slack/:orgId	POST	Set Slack webhook URL
/v1/audit	GET	Audit event stream (paginated)

A.3.6 Benchmarks

Endpoint	Method	Purpose
/v1/benchmark/contribute	POST	Opt in and contribute snapshot
/v1/benchmark/percentiles	GET	Cohort percentiles (404 if k<5)
/v1/benchmark/summary	GET	Summary across metrics

A.4 Environment Variables & Secrets

Name	Type	Purpose
ENVIRONMENT	var	production / preview / development
ALLOWED_ORIGINS	var	CORS allowlist (supports *. wildcard suffix)
RATE_LIMIT_RPM	var	Default 1000, per-org per-minute
RESEND_API_KEY	secret	Email sending (Resend)
COPILOT_ENC_KEY	secret	AES-256-GCM key for GitHub Copilot tokens
DATADOG_ENC_KEY	secret	AES-256-GCM key for Datadog API keys

A.5 Further Reading

The internal research canon for deeper context:

#	Document	When to read
01	CloudHealth → VMware acquisition (TechCrunch, 2018)	Week 1 — best analog
02	Apptio S-1 / IBM acquisition docs (2019)	Month 2 — ITFM / TBM market
03	EU AI Act — Articles 13, 17, 26	Month 3 — governance mandate
04	Sequoia "AI's $600B Question" (2024)	Week 1 — macro tailwind
05	GitHub Copilot Productivity Research (Microsoft/MIT, 2023)	Week 2 — productivity gap
06	Helicone / LangSmith / Langfuse GitHub changelogs	Ongoing — watch for semantic cache
07	OpenTelemetry Semantic Conventions for GenAI (CNCF, 2024)	Week 3 — OTel conformance
08	a16z "The New Language Model Stack" (2023)	Month 1 — layer dynamics
09	Chip Huyen — "AI Engineering" (2024 book)	Month 2 — production LLM systems
10	Paul Graham — "Do Things That Don't Scale" (YC)	Week 1 — design-partner outreach

A.6 Cross-Reference Index

Quick jump to any chapter:

→ Chapter 1 — Product Overview & North Star
→ Chapter 2 — Product Strategy & 18-Month Roadmap
→ Chapter 3 — System Architecture
→ Chapter 4 — Data Model
→ Chapter 5 — Authentication & Authorization
→ Chapter 6 — Event Ingest Pipeline
→ Chapter 7 — Analytics Engine
→ Chapter 8 — Real-Time
→ Chapter 9 — Client Integration Patterns
→ Chapter 10 — Cross-Platform Console & Adapters
→ Chapter 11 — Semantic Caching
→ Chapter 12 — Vantage Agent CLI
→ Chapter 13 — MCP Server & Local Proxy
→ Chapter 14 — Security Model
→ Chapter 15 — CI/CD & Operational Runbook
→ Chapter 16 — Competitive Landscape & Moat

End of guidebook · Vantage AI · Kamal Soft Pvt Ltd · Confidential · April 2026